

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
BỘ MÔN ĐIỀU KHIỂN HỌC**



ĐỒ ÁN TỐT NGHIỆP
CHUYÊN NGÀNH: TỰ ĐỘNG HÓA

Đề Tài:

**NGHIÊN CỨU, THIẾT KẾ VÀ CHẾ TẠO MÔ HÌNH XE ROBOT TỰ
HÀNH PHÁT HIỆN VÀ DẬP TẮT ĐÁM CHÁY SỬ DỤNG ESP32,
RASPBERRY PI 4 VÀ CẢM BIẾN LIDAR**

Sinh viên thực hiện:	Nguyễn Hữu Hòa
Mã sinh viên:	211602310
Lớp:	Kĩ sư Tự động hóa 1 – K62
Giảng viên hướng dẫn:	TS. Phí Văn Lâm

Hà Nội – 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
BỘ MÔN ĐIỀU KHIỂN HỌC**



ĐỒ ÁN TỐT NGHIỆP
CHUYÊN NGÀNH: TỰ ĐỘNG HÓA

Đề Tài:

**NGHIÊN CỨU, THIẾT KẾ VÀ CHẾ TẠO MÔ HÌNH XE ROBOT TỰ
HÀNH PHÁT HIỆN VÀ DẬP TẮT ĐÁM CHÁY SỬ DỤNG ESP32,
RASPBERRY PI 4 VÀ CẢM BIẾN LIDAR**

Sinh viên thực hiện:	Nguyễn Hữu Hòa
Mã sinh viên:	211602310
Lớp:	Kĩ sư Tự động hóa 1 – K62
Giảng viên hướng dẫn:	TS. Phí Văn Lâm

MỤC LỤC

MỤC LỤC	2
DANH MỤC CÁC HÌNH ẢNH	5
DANH MỤC CÁC BẢNG BIỂU	6
LỜI CẢM ƠN	7
LỜI MỞ ĐẦU	8
CHƯƠNG 1. TỔNG QUAN VỀ XE TỰ HÀNH CỨU HỎA	9
1.1. Giới thiệu chung	9
1.2. Phân tích yêu cầu hệ thống	11
1.2.1. Yêu cầu chức năng	11
1.2.2. Yêu cầu kỹ thuật	11
1.3. Nguyên lý hoạt động	11
1.4. Tổng quan về công nghệ Lidar	12
1.5. Tổng quan về thuật toán SLAM	14
1.6. Nền tảng ROS trong xe tự hành	16
CHƯƠNG 2. THIẾT KẾ PHẦN CỨNG CHO XE TỰ HÀNH CỨU HỎA	20
2.1. Sơ đồ tổng quát hệ thống	20
2.2. Lựa chọn thiết bị	20
2.2.1. Thiết kế khung xe	21
2.2.2. Lựa chọn động cơ DC	22
2.2.3. Lựa chọn mạch cầu H	23
2.2.4. Vi điều khiển ESP32	26
2.2.5. Raspberry Pi 4	28
2.2.6. Cảm biến RPLidar A2M12	29
2.2.7. Cảm biến lửa	31
2.2.8. Servo SG90	32
2.2.9. Động cơ bơm nước	32
2.2.9. Nguồn	33
2.2.10. Tay cầm PS3 Bluetooth	34
2.2.11. Sơ đồ đấu nối	35
CHƯƠNG 3: XÂY DỰNG THUẬT TOÁN ĐIỀU KHIỂN CHUYỂN ĐỘNG CHO ROBOT	36
3.1. Điều khiển động cơ DC sử dụng thuật toán PID	36

3.2. Xác định vị trí và định hướng robot	37
3.3. Thuật toán xây dựng bản đồ bằng SLAM Toolbox	38
3.4. Lưu đồ thuật toán	44
3.5. Kết quả xây dựng mô hình hệ thống xe tự hành cứu hỏa	47
3.5.1. Kết quả xây dựng mô hình phần cứng	47
3.5.2. Kết quả thực nghiệm	48
KẾT LUẬN KIẾN NGHỊ VÀ HƯỚNG PHÁT TRIỂN	50
TÀI LIỆU THAM KHẢO	52
PHỤ LỤC	53

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Ý nghĩa
ROS	Robot Operating System	Hệ điều hành robot mã nguồn mở.
LiDAR	Light Detection and Ranging	Công nghệ đo khoảng cách bằng tia laser.
SLAM	Simultaneous Localization and Mapping	Định vị và xây dựng bản đồ đồng thời.
PID	Proportional-Integral-Derivative	Bộ điều khiển tỉ lệ - tích phân - vi phân.
PWM	Pulse Width Modulation	Điều chế độ rộng xung.
UART	Universal Asynchronous Receiver-Transmitter	Giao tiếp nối tiếp không đồng bộ.
BLE	Bluetooth Low Energy	Bluetooth năng lượng thấp.

DANH MỤC CÁC HÌNH ẢNH

Hình 1 . Công nghệ về LiDAR.....	13
Hình 2 . Tổng quan simultaneous localization and mapping.....	14
Hình 3 . Bản đồ 3D được xây dựng từ thuật toán SLAM.....	16
Hình 4 . Các thành phần cơ bản của ROS.....	17
Hình 5 . Nền tảng ROS trong xe tự hành.....	18
Hình 6 . Sơ đồ khối tổng thể hệ thống xe tự hành cứu hỏa.....	20
Hình 7 . Khung tầng 1 của xe.....	21
Hình 8 . Khung tầng 2 của xe.....	21
Hình 9 . Khung tầng 3 của xe.....	22
Hình 10 . Toàn bộ khung xe sau khi lắp ráp.....	22
Hình 11 . Động cơ DC JGA25-370.....	23
Hình 12 . Sơ đồ nguyên lý cầu H.....	24
Hình 13 . Module điều khiển động cơ L298N.....	25
Hình 14 . Chức năng các chân của Module điều khiển động cơ L298N.....	25
Hình 15 . Vi xử lý ESP32.....	27
Hình 16 . Sơ đồ chân của ESP32.....	27
Hình 17 . Máy tính nhúng Raspberry Pi 4.....	29
Hình 18 . Cảm biến RPLiDAR A2M12.....	30
Hình 19 . Sơ đồ chân cảm biến RPLiDAR A2M12.....	31
Hình 20 . Cảm biến lửa.....	32
Hình 21 . Động cơ servo SG90S.....	32
Hình 22 . Động cơ bơm nước mini 12V.....	33
Hình 23 . Pin 18650.....	33
Hình 24 . Mạch hạ áp LM2596.....	34
Hình 25 . Mạch hạ áp XL4015.....	34
Hình 26 . Tay cầm PS3 Bluetooth.....	35
Hình 27 . Sơ đồ nguyên lý mạch điều khiển.....	35
Hình 28 . Sơ đồ nguyên lý mạch công suất.....	35
Hình 29 . Sơ đồ PID.....	36
Hình 30 . Sơ đồ tín hiệu từ Lidar.....	38
Hình 31 . Dữ liệu scan từ Lidar hiển thị trên Rviz.....	39
Hình 32 . Mối liên hệ giữa base link và base laser.....	41
Hình 33 . Bản đồ sau khi Scan Matching.....	42
Hình 34 . Ô lưới trên bản đồ.....	42
Hình 35 . Bản đồ quét được Lidar.....	44
Hình 36 . Lưu đồ thuật toán.....	46
Hình 37 . Hình ảnh kết quả xây dựng mô hình phần cứng robot.....	47
Hình 38 . Hình ảnh khi robot phát hiện ngọn lửa.....	47
Hình 39 . Hình ảnh kết quả xây dựng bản đồ của robot trên Rviz2.....	48

DANH MỤC CÁC BẢNG BIỂU

<i>Bảng 1 . Các thiết bị cho xe tự hành cứu hỏa.....</i>	<i>20</i>
<i>Bảng 2 . Thông số động cơ DC JGA25-370.....</i>	<i>23</i>
<i>Bảng 3 . Thông số kỹ thuật của Module điều khiển động cơ L298N.....</i>	<i>25</i>
<i>Bảng 4 . Chức năng các chân của Module điều khiển động cơ L298N.....</i>	<i>26</i>
<i>Bảng 5 . Thông số kỹ thuật của ESP32.....</i>	<i>27</i>
<i>Bảng 6 . Chức năng các chân của ESP32.....</i>	<i>27</i>
<i>Bảng 7 . Thông số kỹ thuật Raspberry Pi 4.....</i>	<i>29</i>
<i>Bảng 8 . Thông số kỹ thuật cảm biến LiDAR A2M12.....</i>	<i>30</i>
<i>Bảng 9 . Chức năng các chân của cảm biến RPLiDAR A2M12.....</i>	<i>31</i>

LỜI CẢM ƠN

Đồ án tốt nghiệp là dấu mốc khép lại hành trình học tập tại giảng đường đại học, đồng thời là cầu nối giúp sinh viên vận dụng lý thuyết vào giải quyết bài toán thực tế. Để hoàn thành chặng đường ý nghĩa này, em đã nhận được sự đồng hành và hỗ trợ vô cùng lớn và nhiệt tình đến từ các thầy cô khoa Điện – Điện Tử.

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc tới thầy **TS Phí Văn Lâm**. Trong suốt quá trình triển khai đề tài đồ án tốt nghiệp, thầy đã luôn dành thời gian định hướng, tận tình hướng dẫn và góp ý cho em từ lúc bắt đầu cho đến khi hoàn thiện báo cáo. Những góp ý nghiêm túc, khắt khe và tỉ mỉ của thầy đã giúp em rèn luyện tư duy logic và là hành trang quý báu để em phát triển con đường công việc sau này.

Em cũng xin gửi lời cảm ơn tới ban giám hiệu nhà trường cùng toàn thể quý thầy, cô thuộc khoa **Điện – Điện tử**. Những kiến thức chuyên môn và sự giảng dạy tận tâm của thầy cô trong những năm học qua đã xây dựng cho em một nền tảng kiến thức vững chắc cũng như kỹ năng mềm để em tự tin đối mặt với những khó khăn trong quá trình thực hiện đề tài.

Dù đã cố gắng hoàn thành đề tài đồ án tốt nghiệp với tinh thần trách nhiệm cao nhất, tuy nhiên do giới hạn về mặt thời gian và kinh nghiệm, đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp, nhận xét từ quý thầy cô để đồ án được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Hà Nội, ngày 13 tháng 6 năm 2026

Sinh viên

Nguyễn Hữu Hòa

LỜI MỞ ĐẦU

Trong bối cảnh xã hội hiện đại, cùng với sự phát triển nhanh chóng của các khu công nghiệp, khu dân cư và các công trình xây dựng quy mô lớn, nguy cơ xảy ra cháy nổ ngày càng gia tăng đáng kể cả về tần suất lẫn mức độ nghiêm trọng. Các vụ cháy lớn không chỉ gây thiệt hại nặng nề về tài sản mà còn đe dọa trực tiếp đến tính mạng con người, đặc biệt là lực lượng phòng cháy chữa cháy khi phải làm việc trong môi trường nguy hiểm như nhiệt độ cao, khói độc dày đặc và nguy cơ sập đổ công trình.

Trong thực tế, việc tiếp cận đám cháy gặp nhiều khó khăn do tầm nhìn bị hạn chế bởi khói, địa hình phức tạp hoặc không gian kín. Điều này đặt ra yêu cầu cần có các giải pháp công nghệ nhằm hỗ trợ hoặc thay thế con người trong những tình huống nguy hiểm. Một trong những hướng nghiên cứu nổi bật hiện nay là ứng dụng robot tự hành vào công tác chữa cháy nhằm nâng cao hiệu quả cứu hộ và phát hiện nguồn cháy.

Là sinh viên chuyên ngành Tự động hóa, sau quá trình học tập và nghiên cứu các kiến thức chuyên môn trên giảng đường, đồng thời tìm hiểu những ứng dụng thực tiễn của công nghệ điều khiển tự động trong đời sống và sản xuất, em nhận thấy việc phát triển các hệ thống robot tự hành có khả năng giám sát, phát hiện và xử lý các tình huống nguy hiểm là một hướng nghiên cứu có ý nghĩa khoa học và tính ứng dụng cao.

Đặc biệt, trong lĩnh vực phòng cháy chữa cháy, việc ứng dụng các cảm biến thông minh, công nghệ định vị và điều hướng tự động kết hợp với các bộ điều khiển nhúng giúp nâng cao hiệu quả phát hiện và xử lý đám cháy, đồng thời giảm thiểu rủi ro cho con người trong các môi trường nguy hiểm đòi hỏi khả năng hoạt động ổn định.

Xuất phát từ mong muốn vận dụng những kiến thức đã được học để thiết kế và chế tạo một hệ thống robot có khả năng tự hành, nhận biết môi trường và thực hiện nhiệm vụ chữa cháy tự động, em đã quyết định lựa chọn đề nghiên cứu và phát triển thực tế đề tài: **“Nghiên cứu, thiết kế và chế tạo mô hình xe robot tự hành phát hiện và dập tắt đám cháy sử dụng ESP32, Raspberry Pi 4 và cảm biến LiDAR”**.

Trong quá trình nghiên cứu và thực hiện đồ án, do kiến thức và kinh nghiệm thực tế của bản thân còn nhiều hạn chế nên đồ án không tránh khỏi những sai sót. Em rất mong nhận được sự góp ý và chỉ bảo quý báu của quý thầy cô để đề tài được hoàn thiện hơn, đồng thời giúp em tích lũy thêm kinh nghiệm trên chặng đường công việc trong tương lai.

CHƯƠNG 1. TỔNG QUAN VỀ XE TỰ HÀNH CỨU HỎA

1.1. Giới thiệu chung

Trong bối cảnh xã hội hiện đại, cùng với sự phát triển nhanh chóng của các khu công nghiệp, khu dân cư và các công trình xây dựng quy mô lớn, nguy cơ xảy ra cháy nổ ngày càng gia tăng cả về tần suất lẫn mức độ nghiêm trọng. Các vụ cháy lớn không chỉ gây thiệt hại nặng nề về tài sản mà còn đe dọa trực tiếp đến tính mạng con người, đặc biệt là lực lượng phòng cháy chữa cháy khi phải làm việc trong môi trường nguy hiểm như nhiệt độ cao, khói độc dày đặc và nguy cơ sập đổ công trình.

Trong thực tế, việc tiếp cận đám cháy gặp nhiều khó khăn do tầm nhìn bị hạn chế bởi khói, địa hình phức tạp hoặc không gian kín. Điều này đặt ra yêu cầu cần có các giải pháp công nghệ nhằm hỗ trợ hoặc thay thế con người trong những tình huống nguy hiểm. Một trong những hướng nghiên cứu nổi bật hiện nay là ứng dụng robot tự hành vào công tác chữa cháy giúp tăng cao hiệu quả xử lý sự cố.

Xe tự hành cứu hỏa sử dụng cảm biến LiDAR là một hệ thống robot di động có khả năng tự điều hướng, nhận biết môi trường xung quanh và tiếp cận khu vực cháy mà không cần sự can thiệp trực tiếp của con người. Hệ thống này hoạt động dựa trên sự kết hợp giữa các cảm biến, bộ xử lý và thuật toán điều khiển thông minh.

Trong đó, công nghệ LiDAR (Light Detection and Ranging) đóng vai trò cốt lõi. Đây là công nghệ sử dụng tia laser để đo khoảng cách đến các vật thể bằng cách tính toán thời gian phản xạ của tia sáng. Dữ liệu thu được từ LiDAR cho phép xây dựng bản đồ môi trường với độ chính xác cao dưới dạng đám mây điểm. Nhờ đó, robot có thể nhận biết được hình dạng, vị trí và khoảng cách của các vật thể xung quanh.

Khi được tích hợp vào xe tự hành cứu hỏa, LiDAR mang lại nhiều lợi ích quan trọng. Trước hết, cảm biến giúp robot phát hiện và tránh vật cản trong môi trường có nhiều khói, nơi mà các cảm biến quang học thông thường hoạt động kém hiệu quả. Bên cạnh đó, LiDAR còn hỗ trợ xây dựng bản đồ khu vực cháy theo thời gian thực, từ đó giúp robot xác định vị trí của mình và lập kế hoạch di chuyển tối ưu. Ngoài ra, hệ thống còn có khả năng tìm đường đi ngắn nhất và an toàn nhất để tiếp cận nguồn cháy góp phần nâng cao khả năng hoạt động tự động của toàn bộ hệ thống.

Xe tự hành cứu hỏa có thể hoạt động trong nhiều điều kiện khắc nghiệt như môi trường có nhiệt độ cao, khói dày đặc hoặc không gian kín, nơi con người khó hoặc không thể tiếp cận. Nhờ khả năng tự động hóa cao, hệ thống này góp phần giảm thiểu rủi ro cho lực lượng cứu hỏa, đồng thời nâng cao hiệu quả và tốc độ xử lý các tình huống cháy nổ.

Sự phát triển mạnh mẽ của các công nghệ như trí tuệ nhân tạo (AI), Internet vạn vật (IoT), cảm biến thông minh và hệ thống điều khiển nhúng đã tạo điều kiện thuận lợi cho việc nghiên cứu và phát triển các hệ thống robot tự hành phục vụ trong nhiều lĩnh vực khác nhau. Trong đó, robot cứu hộ và chữa cháy đang nhận được sự quan tâm đặc biệt nhờ khả năng làm việc trong những môi trường nguy hiểm mà con người khó tiếp cận. Việc ứng dụng robot vào công tác phòng cháy chữa cháy không chỉ giúp giảm thiểu rủi ro cho lực lượng cứu hỏa mà còn nâng cao hiệu quả xử lý sự cố, đặc biệt trong những tình huống khẩn cấp đòi hỏi phản ứng nhanh.

Trong những năm gần đây, nhiều quốc gia trên thế giới đã nghiên cứu và triển khai các loại robot chữa cháy với mức độ tự động hóa khác nhau. Một số hệ thống có khả năng điều khiển từ xa để tiếp cận khu vực nguy hiểm, trong khi các hệ thống tiên tiến hơn được trang bị khả năng tự hành, tự nhận biết môi trường và tự thực hiện

nhiệm vụ mà không cần sự can thiệp liên tục của con người. Điều này cho thấy xu hướng phát triển tất yếu của các hệ thống robot thông minh trong lĩnh vực cứu hộ và phòng cháy chữa cháy.

Tại Việt Nam, các nghiên cứu về robot chữa cháy cũng đang từng bước được triển khai tại các trường đại học, viện nghiên cứu và doanh nghiệp công nghệ. Tuy nhiên, phần lớn các mô hình hiện nay vẫn tập trung vào chức năng điều khiển từ xa hoặc hoạt động trong môi trường đơn giản. Việc nghiên cứu và phát triển một hệ thống xe tự hành có khả năng tự xây dựng bản đồ môi trường, tự định vị, tránh vật cản và tự động thực hiện nhiệm vụ chữa cháy vẫn là một hướng nghiên cứu có nhiều tiềm năng và ý nghĩa thực tiễn.

Đề tài "Xe tự hành cứu hỏa sử dụng cảm biến LiDAR" được thực hiện nhằm nghiên cứu và ứng dụng các công nghệ hiện đại vào việc xây dựng một mô hình robot chữa cháy thông minh. Hệ thống được thiết kế dựa trên nền tảng Raspberry Pi 4 kết hợp với vi điều khiển ESP32 để đảm nhiệm các nhiệm vụ xử lý dữ liệu, điều khiển động cơ và giao tiếp giữa các thành phần phần cứng. Cảm biến LiDAR được sử dụng để thu thập dữ liệu môi trường phục vụ cho quá trình định vị và lập bản đồ thông qua thuật toán SLAM. Ngoài ra, robot còn được trang bị cảm biến phát hiện lửa, cơ cấu bơm nước và các thuật toán điều hướng nhằm thực hiện nhiệm vụ chữa cháy tự động.

Một trong những điểm nổi bật của hệ thống là khả năng xây dựng bản đồ môi trường theo thời gian thực. Thông qua dữ liệu thu được từ LiDAR, robot có thể nhận biết được cấu trúc không gian xung quanh, xác định vị trí của bản thân và lập kế hoạch di chuyển phù hợp. Điều này giúp hệ thống hoạt động linh hoạt hơn so với các robot chỉ di chuyển theo quỹ đạo được lập trình sẵn. Khi môi trường thay đổi hoặc xuất hiện vật cản mới, robot vẫn có thể cập nhật thông tin và đưa ra quyết định điều hướng phù hợp.

Bên cạnh đó, việc kết hợp giữa thuật toán SLAM và các phương pháp tránh vật cản giúp robot có khả năng di chuyển an toàn trong môi trường phức tạp. Đây là yếu tố đặc biệt quan trọng đối với các nhiệm vụ chữa cháy bởi khu vực xảy ra hỏa hoạn thường chứa nhiều vật cản, khói bụi và các điều kiện làm việc không ổn định. Khả năng nhận biết môi trường và phản ứng theo thời gian thực giúp robot duy trì hoạt động ổn định và nâng cao hiệu quả thực hiện nhiệm vụ.

Ngoài giá trị ứng dụng trong lĩnh vực phòng cháy chữa cháy, các công nghệ được nghiên cứu trong đề tài còn có thể mở rộng sang nhiều lĩnh vực khác như robot tuần tra, robot vận chuyển hàng hóa trong nhà kho, robot hỗ trợ cứu hộ hoặc các hệ thống tự hành trong công nghiệp. Do đó, việc nghiên cứu và xây dựng mô hình xe tự hành cứu hỏa không chỉ mang ý nghĩa thực tiễn mà còn góp phần tạo nền tảng cho việc phát triển các hệ thống robot thông minh trong tương lai.

Từ những yêu cầu thực tế và xu hướng phát triển của công nghệ hiện nay, việc nghiên cứu xe tự hành cứu hỏa sử dụng cảm biến LiDAR là cần thiết và có ý nghĩa khoa học cũng như giá trị ứng dụng cao. Đề tài không chỉ góp phần nâng cao khả năng ứng dụng công nghệ tự hành vào lĩnh vực phòng cháy chữa cháy mà còn tạo tiền đề cho các nghiên cứu chuyên sâu hơn về robot tự hành và trí tuệ nhân tạo trong tương lai.

1.2. Phân tích yêu cầu hệ thống

Mục tiêu của đề tài là nghiên cứu, thiết kế và chế tạo mô hình xe robot tự hành có khả năng phát hiện và dập tắt đám cháy trong môi trường trong nhà. Hệ thống phải đáp ứng các yêu cầu về khả năng tự hành, nhận biết môi trường, phát hiện nguồn lửa và thực hiện hành động chữa cháy một cách tự động.

1.2.1. Yêu cầu chức năng

Hệ thống robot được thiết kế cần đáp ứng các chức năng chính sau:

- + Tự động di chuyển trong môi trường làm việc.
- + Thu thập dữ liệu môi trường bằng cảm biến LiDAR để nhận biết vật cản xung quanh.
- + Xây dựng bản đồ môi trường và xác định vị trí của robot thông qua thuật toán SLAM.
- + Tự động tránh vật cản trong quá trình di chuyển.
- + Phát hiện đám cháy thông qua cảm biến phát hiện lửa.
- + Xác định hướng và vị trí tương đối của nguồn cháy.
- + Kích hoạt bơm nước hoặc cơ cấu chữa cháy để dập tắt đám cháy.
- + Truyền và nhận dữ liệu giữa Raspberry Pi 4 và ESP32 thông qua giao tiếp UART.
- + Hiển thị thông tin trạng thái hoạt động của robot trên máy tính.

1.2.2. Yêu cầu kỹ thuật

Để đảm bảo khả năng hoạt động ổn định, hệ thống cần đáp ứng các yêu cầu kỹ thuật sau:

- + Robot phải hoạt động liên tục trong thời gian tối thiểu 30 phút.
- + Sai số định vị phải nằm trong giới hạn cho phép khi thực hiện lập bản đồ và di chuyển.
- + LiDAR phải có khả năng quét 360° để thu thập đầy đủ dữ liệu môi trường.
- + Tốc độ xử lý dữ liệu phải đáp ứng yêu cầu điều hướng thời gian thực.
- + Hệ thống truyền thông giữa Raspberry Pi 4 và ESP32 phải ổn định và có độ trễ thấp.
- + Cơ cấu bơm nước phải đủ lưu lượng để mô phỏng quá trình dập tắt đám cháy.
- + Robot phải có khả năng phát hiện và tránh va chạm với vật cản trong phạm vi hoạt động.

1.3. Nguyên lý hoạt động

Quá trình xây dựng một hệ thống xe tự hành cứu hỏa sử dụng LiDAR là một quá trình phức tạp, bao gồm nhiều bước từ thu thập dữ liệu, xử lý thông tin đến điều khiển robot và thực hiện nhiệm vụ chữa cháy.

Đầu tiên là quá trình thu thập và xử lý dữ liệu. Cảm biến LiDAR được sử dụng để quét môi trường xung quanh và thu thập dữ liệu khoảng cách dưới dạng các điểm trong không gian. Tuy nhiên, dữ liệu thu được thường chứa nhiều nhiễu do ảnh hưởng của môi trường, vì vậy cần phải thực hiện các bước tiền xử lý như lọc nhiễu, loại bỏ dữ liệu không hợp lệ và chuẩn hóa dữ liệu trước khi đưa vào các thuật toán xử lý tiếp theo.

Sau đó, dữ liệu được sử dụng để xây dựng bản đồ môi trường thông qua các thuật toán SLAM (Simultaneous Localization and Mapping). Đây là kỹ thuật cho phép robot vừa xây dựng bản đồ môi trường, vừa xác định vị trí của chính nó trong bản đồ đó. Tùy vào yêu cầu, bản đồ có thể được xây dựng dưới dạng 2D hoặc 3D.

Tiếp theo là quá trình định vị robot. Dựa trên dữ liệu cảm biến và bản đồ đã xây dựng, robot xác định vị trí hiện tại của mình trong không gian. Các thuật toán như

Kalman Filter hoặc Particle Filter thường được sử dụng để cải thiện độ chính xác và giảm sai số trong quá trình định vị.

Trong quá trình di chuyển, robot cần liên tục điều khiển và tránh vật cản. Dựa trên dữ liệu cảm biến thời gian thực, hệ thống điều chỉnh tốc độ và hướng di chuyển của robot nhằm đảm bảo an toàn và chính xác. Các thuật toán điều khiển như PID có thể được sử dụng để đảm bảo chuyển động ổn định. Nhờ đó, robot có thể di chuyển linh hoạt trong môi trường phức tạp và tiếp cận khu vực xảy ra cháy một cách hiệu quả.

Cuối cùng là quá trình thực hiện nhiệm vụ chữa cháy. Khi robot tiếp cận được khu vực có nguồn nhiệt, cảm biến lửa sẽ xác định chính xác vị trí đám cháy. Hệ thống điều khiển sẽ kích hoạt bơm nước hướng vòi phun để dập lửa một cách hiệu quả.

1.4. Tổng quan về công nghệ Lidar

LiDAR (Light Detection and Ranging) là công nghệ đo khoảng cách và thu thập thông tin không gian bằng cách sử dụng tia laser. Đây là một trong những công nghệ cảm biến hiện đại được ứng dụng rộng rãi trong các lĩnh vực như xe tự hành, robot di động, khảo sát địa hình, bản đồ số, nông nghiệp thông minh và hệ thống giám sát môi trường. Với khả năng cung cấp dữ liệu khoảng cách có độ chính xác cao trong thời gian thực, LiDAR đã trở thành một thành phần quan trọng trong các hệ thống robot tự hành hiện nay.

Nguyên lý hoạt động của LiDAR dựa trên phương pháp đo thời gian bay của tia laser (Time of Flight - ToF). Cảm biến sẽ phát ra các xung laser theo nhiều hướng khác nhau trong không gian. Khi tia laser gặp vật thể, tín hiệu sẽ phản xạ trở lại bộ thu của cảm biến. Bằng cách đo khoảng thời gian từ lúc phát đến lúc nhận tín hiệu phản xạ và biết được vận tốc truyền của ánh sáng, hệ thống có thể tính toán chính xác khoảng cách từ cảm biến đến vật thể.

Trong đó, (d) là khoảng cách đến vật thể, (c) là vận tốc ánh sáng và (t) là thời gian truyền tín hiệu. Quá trình này được thực hiện liên tục với tần số rất cao, cho phép LiDAR thu thập hàng nghìn đến hàng triệu điểm đo trong mỗi giây.

Dữ liệu thu được từ LiDAR thường được biểu diễn dưới dạng đám mây điểm (Point Cloud). Mỗi điểm trong đám mây điểm chứa thông tin về vị trí của vật thể trong không gian tương ứng với khoảng cách và góc quét của tia laser. Khi tập hợp nhiều điểm đo lại với nhau, hệ thống có thể tái tạo hình dạng của môi trường xung quanh một cách chi tiết. Đây là nguồn dữ liệu quan trọng để robot nhận biết cấu trúc môi trường, xác định vị trí vật cản và xây dựng bản đồ phục vụ cho quá trình điều hướng.

Hiện nay, LiDAR được chia thành nhiều loại khác nhau dựa trên cấu tạo và nguyên lý quét. LiDAR 2D là loại phổ biến trong các hệ thống robot di động trong nhà do có giá thành hợp lý và khả năng xử lý dữ liệu đơn giản. Loại cảm biến này quét trên một mặt phẳng ngang với góc quét có thể đạt tới 360 độ. Trong khi đó, LiDAR 3D có khả năng thu thập dữ liệu không gian theo nhiều lớp quét khác nhau, cho phép tái tạo môi trường ba chiều với độ chi tiết cao hơn. Tuy nhiên, chi phí của LiDAR 3D thường lớn và yêu cầu năng lực xử lý dữ liệu cao hơn đáng kể.

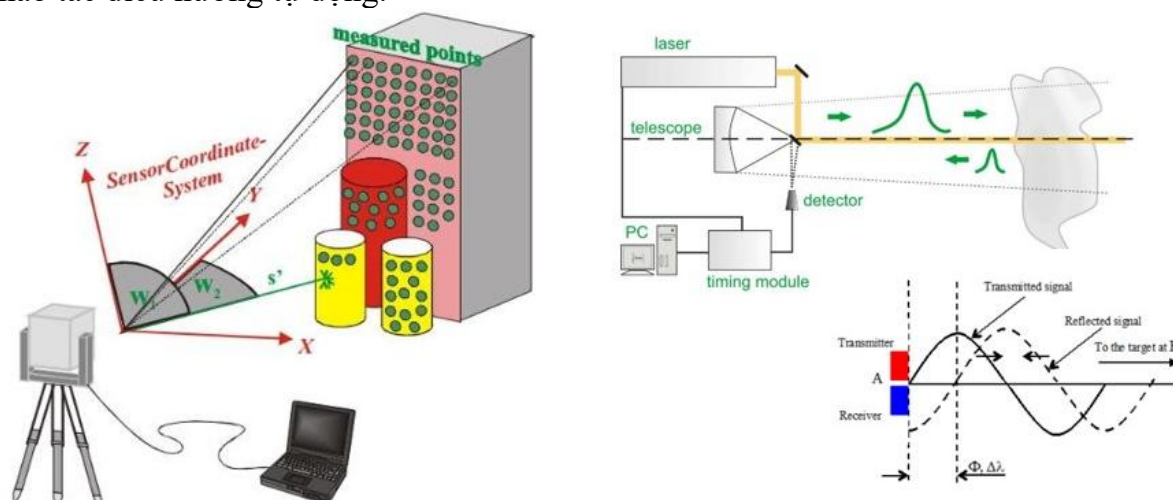
So với các loại cảm biến khoảng cách khác như cảm biến siêu âm hay cảm biến hồng ngoại, LiDAR sở hữu nhiều ưu điểm nổi bật. Cảm biến siêu âm có cấu trúc đơn giản và giá thành thấp nhưng phạm vi hoạt động hạn chế, độ phân giải thấp và dễ bị ảnh hưởng bởi điều kiện môi trường. Trong khi đó, cảm biến hồng ngoại có khả năng phát hiện vật thể ở khoảng cách gần nhưng thường gặp khó khăn khi làm việc dưới điều kiện ánh sáng mạnh hoặc với các bề mặt phản xạ kém. Ngược lại, LiDAR có thể đo khoảng cách với độ chính xác cao, phạm vi hoạt động rộng và khả năng thu thập dữ

liệu môi trường chi tiết hơn nhiều. Điều này giúp robot có thể nhận biết được hình dạng, kích thước và vị trí của các vật thể xung quanh một cách chính xác.

Một ưu điểm quan trọng khác của LiDAR là khả năng hoạt động ổn định trong điều kiện ánh sáng thay đổi. Không giống như camera phụ thuộc nhiều vào nguồn sáng môi trường, LiDAR sử dụng nguồn laser chủ động nên vẫn có thể thu thập dữ liệu hiệu quả trong điều kiện thiếu sáng hoặc ban đêm. Đây là đặc điểm rất phù hợp đối với các ứng dụng robot cứu hộ và chữa cháy, nơi môi trường làm việc thường có nhiều khói bụi và ánh sáng không ổn định.

Trong lĩnh vực robot tự hành, LiDAR được sử dụng như một cảm biến chủ đạo để hỗ trợ các chức năng nhận biết môi trường, phát hiện vật cản, định vị và lập bản đồ. Dữ liệu từ LiDAR được xử lý thông qua các thuật toán như SLAM (Simultaneous Localization and Mapping) nhằm xây dựng bản đồ môi trường đồng thời xác định vị trí hiện tại của robot. Nhờ đó, robot có thể tự động lập kế hoạch đường đi, tránh chướng ngại vật và di chuyển đến mục tiêu một cách hiệu quả mà không cần sự can thiệp trực tiếp của con người.

Đối với hệ thống xe tự hành cứu hỏa trong đề tài này, LiDAR đóng vai trò là cảm biến chính trong việc thu thập dữ liệu môi trường xung quanh. Trong quá trình hoạt động, cảm biến liên tục quét khu vực làm việc và gửi dữ liệu khoảng cách về bộ xử lý Raspberry Pi. Dữ liệu này được sử dụng để xây dựng bản đồ môi trường thông qua thuật toán SLAM Toolbox trên nền tảng ROS2. Đồng thời, hệ thống cũng sử dụng thông tin từ LiDAR để xác định vị trí của robot, nhận biết vật cản và thực hiện các thao tác điều hướng tự động.



Hình 1. Công nghệ về LiDAR

Khi robot di chuyển trong môi trường có nhiều vật cản như tường, bàn ghế hoặc các chướng ngại vật khác, dữ liệu LiDAR giúp hệ thống nhận biết khoảng cách an toàn và đưa ra các quyết định điều khiển phù hợp nhằm tránh va chạm. Khả năng quét liên tục theo thời gian thực cho phép robot phản ứng nhanh với những thay đổi của môi trường, từ đó nâng cao độ an toàn và độ tin cậy trong quá trình hoạt động.

Trong các tình huống cháy nổ thực tế, môi trường thường xuất hiện nhiều khói bụi và tầm nhìn bị hạn chế. Những điều kiện này có thể làm giảm hiệu quả hoạt động của camera hoặc các cảm biến quang học thông thường. Tuy nhiên, LiDAR vẫn có thể duy trì khả năng đo khoảng cách và nhận biết vật cản ở mức độ nhất định, giúp robot tiếp tục thực hiện nhiệm vụ điều hướng và tiếp cận khu vực xảy ra cháy. Điều này góp phần giảm thiểu rủi ro cho lực lượng cứu hỏa và nâng cao khả năng tự động hóa của hệ thống.

Nhờ những ưu điểm về độ chính xác, khả năng thu thập dữ liệu môi trường chi tiết và hỗ trợ hiệu quả cho các thuật toán định vị, lập bản đồ và điều hướng, LiDAR được xem là một trong những công nghệ quan trọng nhất trong các hệ thống robot tự hành hiện đại. Việc ứng dụng LiDAR trong xe tự hành cứu hỏa không chỉ giúp nâng cao khả năng nhận biết môi trường mà còn tạo nền tảng cho việc phát triển các hệ thống chữa cháy thông minh trong tương lai.

1.5. Tổng quan về thuật toán SLAM

SLAM (Simultaneous Localization and Mapping) là kỹ thuật cho phép robot thực hiện đồng thời hai nhiệm vụ quan trọng là xây dựng bản đồ môi trường (Mapping) và xác định vị trí của chính nó trong bản đồ đó (Localization). Đây được xem là một trong những công nghệ nền tảng của robot tự hành hiện đại, bởi khả năng hoạt động độc lập trong môi trường chưa biết trước mà không cần sự hỗ trợ từ các hệ thống định vị bên ngoài.



Hình 2. Tổng quan simultaneous localization and mapping

Trong thực tế, để một robot có thể tự động di chuyển đến mục tiêu, hệ thống cần biết được hai thông tin quan trọng: robot đang ở đâu và môi trường xung quanh có những gì. Hai bài toán này có mối liên hệ chặt chẽ với nhau. Nếu không có bản đồ môi trường, robot khó có thể xác định chính xác vị trí của mình. Ngược lại, nếu không biết vị trí hiện tại thì việc xây dựng bản đồ cũng sẽ không chính xác. Thuật toán SLAM được phát triển nhằm giải quyết đồng thời hai vấn đề trên bằng cách sử dụng dữ liệu từ các cảm biến để liên tục cập nhật cả bản đồ và vị trí robot theo thời gian thực.

Trong quá trình hoạt động, robot liên tục thu thập dữ liệu từ các cảm biến như LiDAR, camera, cảm biến độ sâu hoặc IMU (Inertial Measurement Unit). Các dữ liệu này được gửi đến thuật toán SLAM để phân tích và xử lý. Dựa trên các đặc trưng của

môi trường được cảm biến ghi nhận, hệ thống sẽ xác định sự thay đổi vị trí của robot qua từng thời điểm và đồng thời cập nhật bản đồ môi trường xung quanh.

Nguyên lý cơ bản của SLAM bao gồm ba bước chính. Bước đầu tiên là thu thập dữ liệu cảm biến từ môi trường. Dữ liệu này có thể là khoảng cách đo được từ LiDAR, hình ảnh từ camera hoặc thông tin gia tốc và góc quay từ IMU. Bước thứ hai là ước lượng vị trí hiện tại của robot dựa trên dữ liệu cảm biến và thông tin chuyển động.

Cuối cùng, hệ thống sử dụng các dữ liệu thu thập được để cập nhật hoặc mở rộng bản đồ môi trường. Quá trình này được lặp lại liên tục trong suốt thời gian robot hoạt động.

Một trong những thách thức lớn nhất của bài toán SLAM là sai số tích lũy trong quá trình di chuyển. Khi robot di chuyển trong thời gian dài, các sai số từ cảm biến và hệ thống điều khiển có thể khiến vị trí ước lượng ngày càng lệch so với thực tế. Để khắc phục vấn đề này, các thuật toán SLAM hiện đại thường sử dụng kỹ thuật Loop Closure. Kỹ thuật này cho phép robot nhận biết khi quay trở lại một khu vực đã đi qua trước đó và tự động điều chỉnh lại bản đồ cũng như vị trí hiện tại nhằm giảm thiểu sai số tích lũy.

Hiện nay có nhiều phương pháp SLAM khác nhau được phát triển để phù hợp với từng loại cảm biến và ứng dụng cụ thể. Một trong những thuật toán phổ biến là Gmapping, sử dụng bộ lọc hạt (Particle Filter) để xây dựng bản đồ lưới chiếm dụng (Occupancy Grid Map). Gmapping có ưu điểm là dễ triển khai và phù hợp với các hệ thống robot sử dụng LiDAR 2D. Tuy nhiên, thuật toán này có thể gặp khó khăn khi robot hoạt động trong môi trường rộng hoặc có nhiều thay đổi.

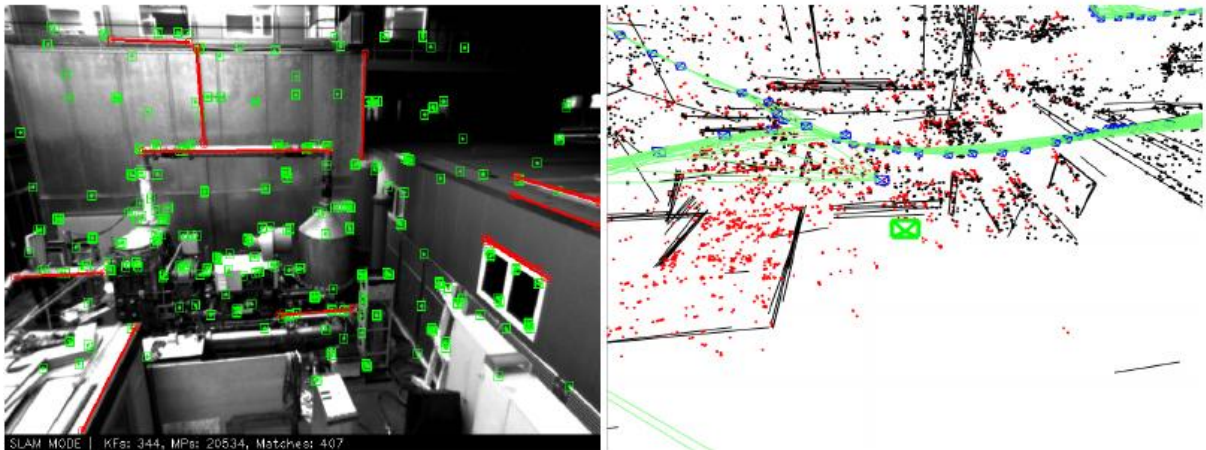
Một phương pháp khác là Hector SLAM, được thiết kế để hoạt động hiệu quả với dữ liệu LiDAR có tần số quét cao. Hector SLAM không phụ thuộc nhiều vào dữ liệu encoder bánh xe nên thường được sử dụng trên các robot có hệ thống đo quãng đường chưa thực sự chính xác. Tuy nhiên, độ ổn định của thuật toán có thể giảm khi robot di chuyển trong môi trường có ít đặc trưng hoặc không gian rộng lớn.

Cartographer là thuật toán SLAM được phát triển bởi Google với khả năng xử lý cả dữ liệu 2D và 3D. Thuật toán này sử dụng kỹ thuật tối ưu hóa đồ thị (Graph Optimization) giúp nâng cao độ chính xác của bản đồ và vị trí robot. Cartographer cho kết quả tốt trong nhiều môi trường khác nhau nhưng yêu cầu tài nguyên tính toán tương đối lớn.

Đối với ROS2, một trong những giải pháp được sử dụng phổ biến hiện nay là SLAM Toolbox. Đây là gói phần mềm được phát triển nhằm hỗ trợ các hệ thống robot sử dụng LiDAR 2D xây dựng bản đồ và định vị trong thời gian thực. SLAM Toolbox tích hợp nhiều tính năng như xây dựng bản đồ trực tuyến (Online Mapping), lưu và tải bản đồ, tối ưu hóa bản đồ bằng kỹ thuật Loop Closure và hỗ trợ cập nhật bản đồ trong quá trình hoạt động. Nhờ những ưu điểm này, SLAM Toolbox được lựa chọn sử dụng trong đề tài xe tự hành cứu hỏa.

Trong hệ thống nghiên cứu, dữ liệu quét từ cảm biến LiDAR được truyền đến Raspberry Pi 4 thông qua ROS2. Gói SLAM Toolbox tiếp nhận dữ liệu LaserScan và sử dụng các thuật toán xử lý để xây dựng bản đồ môi trường dưới dạng Occupancy Grid Map. Đồng thời, hệ thống cũng liên tục xác định vị trí hiện tại của robot trong

bản đồ đã xây dựng. Thông tin này được sử dụng làm cơ sở cho các thuật toán điều hướng và tránh vật cản.



Hình 3. Bản đồ 3D được xây dựng từ thuật toán SLAM

Một ưu điểm quan trọng của SLAM là khả năng hoạt động mà không cần tín hiệu GPS. Trong môi trường trong nhà hoặc khu vực có nhiều vật cản như nhà kho, nhà xưởng hoặc công trình đang cháy, tín hiệu GPS thường rất yếu hoặc không thể sử dụng. Nhờ SLAM, robot vẫn có thể xác định vị trí và di chuyển chính xác chỉ dựa trên dữ liệu cảm biến nội bộ. Điều này đặc biệt phù hợp với các hệ thống robot cứu hộ và chữa cháy, nơi yêu cầu khả năng hoạt động độc lập trong những môi trường phức tạp và nguy hiểm.

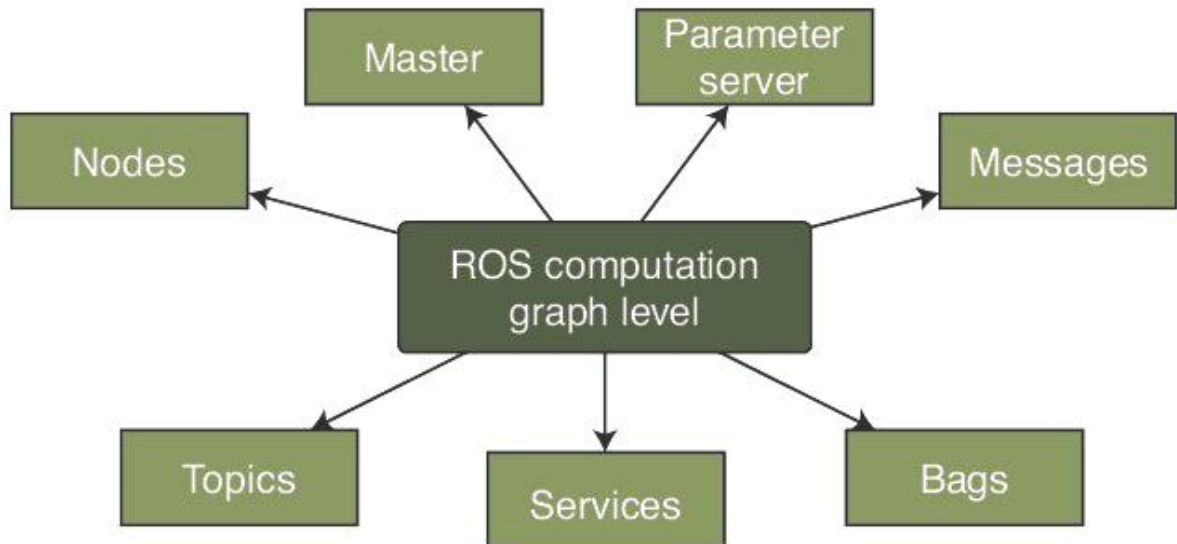
Bên cạnh việc hỗ trợ định vị, SLAM còn đóng vai trò quan trọng trong việc lập kế hoạch đường đi. Khi bản đồ môi trường đã được xây dựng, hệ thống có thể sử dụng các thuật toán tìm đường để xác định lộ trình tối ưu từ vị trí hiện tại đến mục tiêu. Trong đề tài xe tự hành cứu hỏa, bản đồ do SLAM tạo ra giúp robot lựa chọn đường đi phù hợp nhất để tiếp cận đám cháy, đồng thời tránh các vật cản xuất hiện trên đường di chuyển.

Nhờ khả năng xây dựng bản đồ, định vị chính xác và hỗ trợ điều hướng tự động, SLAM đã trở thành một thành phần không thể thiếu trong các hệ thống robot tự hành hiện đại. Việc ứng dụng SLAM Toolbox kết hợp với cảm biến LiDAR trong đề tài không chỉ giúp robot hoạt động ổn định trong môi trường trong nhà mà còn tạo nền tảng cho việc phát triển các chức năng tự hành nâng cao trong tương lai. Qua đó góp phần nâng cao hiệu quả hoạt động của hệ thống xe tự hành cứu hỏa và giảm thiểu sự phụ thuộc vào sự can thiệp trực tiếp của con người.

1.6. Nền tảng ROS trong xe tự hành

Robot Operating System (ROS) là một nền tảng phần mềm mã nguồn mở được phát triển nhằm hỗ trợ việc thiết kế, lập trình và vận hành các hệ thống robot. Mặc dù có tên gọi là "Operating System" (hệ điều hành), ROS thực chất không phải là một hệ điều hành hoàn chỉnh mà là một tập hợp các thư viện, công cụ và giao thức truyền thông được xây dựng trên nền hệ điều hành Linux, phổ biến nhất là Ubuntu. ROS cung cấp một môi trường phát triển thống nhất giúp các nhà nghiên cứu và kỹ sư robot dễ dàng xây dựng các ứng dụng điều khiển robot từ đơn giản đến phức tạp.

ROS được phát triển lần đầu bởi Stanford Artificial Intelligence Laboratory và sau đó được tiếp tục duy trì bởi Willow Garage. Trải qua hơn một thập kỷ phát triển, ROS đã trở thành nền tảng phổ biến nhất trong lĩnh vực robot với cộng đồng người dùng và nhà phát triển rộng lớn trên toàn thế giới. Hiện nay, ROS được ứng dụng trong nhiều lĩnh vực khác nhau như robot công nghiệp, robot dịch vụ, robot tự hành, xe tự lái, robot y tế, robot cứu hộ và các hệ thống nghiên cứu trí tuệ nhân tạo.



Hình 4. Các thành phần cơ bản của ROS

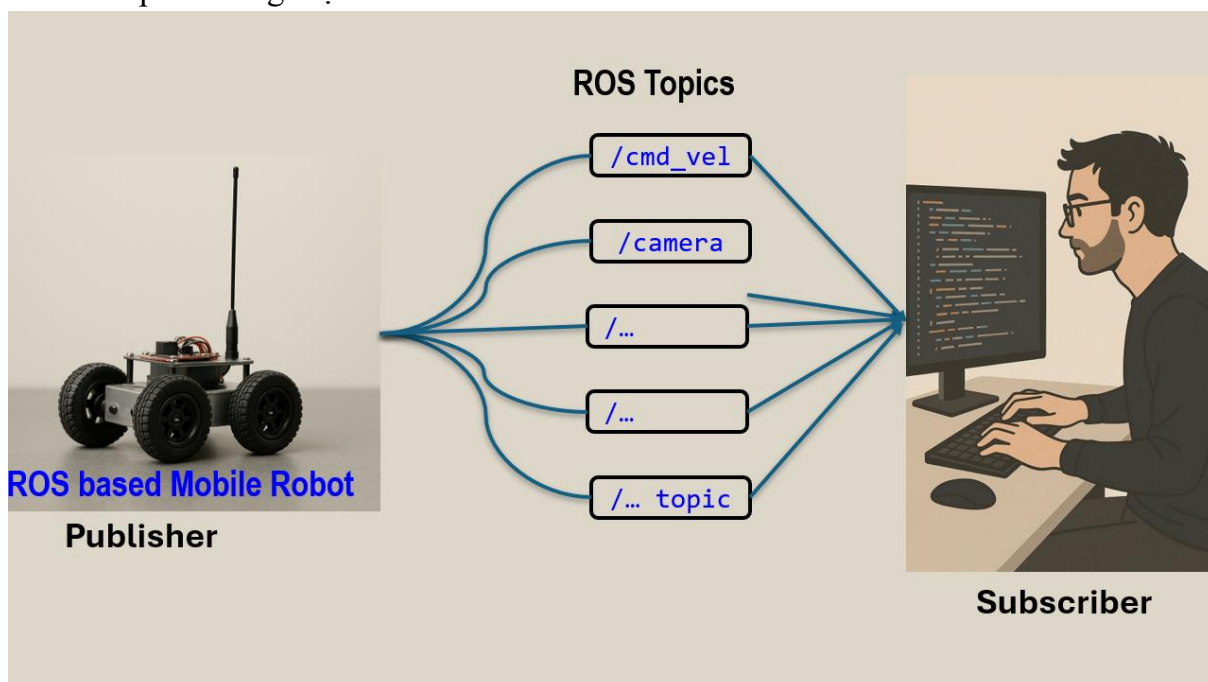
Một trong những ưu điểm nổi bật của ROS là cung cấp sẵn một hệ sinh thái phong phú gồm các thư viện và công cụ hỗ trợ lập trình robot. Các thư viện này bao gồm xử lý dữ liệu cảm biến, điều khiển động cơ, xử lý ảnh, thị giác máy tính, định vị, lập bản đồ, điều hướng tự động và trí tuệ nhân tạo. Nhờ đó, người phát triển có thể tận dụng các gói phần mềm có sẵn thay vì phải xây dựng toàn bộ hệ thống từ đầu, giúp giảm đáng kể thời gian và chi phí phát triển.

Kiến trúc của ROS được xây dựng theo mô hình phân tán (Distributed Architecture). Trong mô hình này, toàn bộ hệ thống được chia thành nhiều thành phần nhỏ gọi là Node. Mỗi node đảm nhận một nhiệm vụ cụ thể trong hệ thống robot. Ví dụ, một node có thể thực hiện việc thu thập dữ liệu từ cảm biến LiDAR, một node khác xử lý thuật toán SLAM để xây dựng bản đồ, trong khi node khác chịu trách nhiệm điều khiển động cơ hoặc điều hướng robot. Việc chia nhỏ hệ thống thành các node độc lập giúp tăng tính linh hoạt, khả năng mở rộng và thuận tiện cho việc bảo trì, nâng cấp hệ thống.

Các node trong ROS giao tiếp với nhau thông qua nhiều cơ chế khác nhau như Topic, Service và Action. Topic là phương thức truyền dữ liệu theo mô hình Publish/Subscribe, trong đó một node phát dữ liệu (Publisher) và các node khác nhận dữ liệu (Subscriber). Đây là cơ chế được sử dụng phổ biến nhất để truyền dữ liệu cảm biến hoặc trạng thái hoạt động của robot theo thời gian thực. Service được sử dụng cho các yêu cầu giao tiếp theo cơ chế hỏi – đáp giữa hai node. Trong khi đó, Action được sử dụng cho các tác vụ kéo dài và cần phản hồi trạng thái thực hiện liên tục.

Một ưu điểm quan trọng của ROS là hỗ trợ nhiều ngôn ngữ lập trình khác nhau. Hai ngôn ngữ được sử dụng phổ biến nhất là C++ và Python. Điều này cho phép các nhà phát triển lựa chọn ngôn ngữ phù hợp với yêu cầu của từng ứng dụng. Ví dụ, các tác vụ yêu cầu hiệu năng cao thường được lập trình bằng C++, trong khi Python được sử dụng để phát triển nhanh các thuật toán thử nghiệm hoặc xử lý dữ liệu.

Bên cạnh các thư viện lập trình, ROS còn cung cấp nhiều công cụ hỗ trợ trực quan hóa và gỡ lỗi hệ thống. Một trong những công cụ nổi bật là RViz, cho phép hiển thị dữ liệu từ cảm biến, bản đồ môi trường, vị trí robot và các đối tượng trong không gian ba chiều. Công cụ này giúp người phát triển dễ dàng theo dõi hoạt động của robot trong thời gian thực. Ngoài ra, ROS còn cung cấp Gazebo – một môi trường mô phỏng mạnh mẽ cho phép xây dựng và kiểm tra robot trong môi trường ảo trước khi triển khai trên phần cứng thực tế.



Hình 5. Nền tảng ROS trong xe tự hành

Để đáp ứng nhu cầu ngày càng cao của các hệ thống robot hiện đại, ROS2 đã được phát triển như một phiên bản nâng cấp của ROS. ROS2 được thiết kế nhằm khắc phục những hạn chế của ROS1, đặc biệt là về hiệu năng, độ ổn định và khả năng hoạt động trong các hệ thống thời gian thực. ROS2 sử dụng giao thức DDS (Data Distribution Service) làm nền tảng truyền thông thay cho ROS Master của ROS1, giúp tăng khả năng mở rộng và cải thiện độ tin cậy trong quá trình trao đổi dữ liệu.

Ngoài ra, ROS2 còn hỗ trợ tốt hơn cho các hệ thống đa robot, các thiết bị nhúng có tài nguyên hạn chế và các ứng dụng công nghiệp yêu cầu tính ổn định cao. Đây là lý do ROS2 đang dần trở thành tiêu chuẩn mới trong lĩnh vực robot hiện đại. Nhiều gói phần mềm quan trọng như Navigation2, SLAM Toolbox và các driver cảm biến mới hiện nay đều được phát triển dựa trên nền tảng ROS2.

Trong đề tài xe tự hành cứu hỏa sử dụng cảm biến LiDAR, ROS2 được lựa chọn làm nền tảng phần mềm chính để điều phối hoạt động của toàn bộ hệ thống. Dữ liệu từ cảm biến LiDAR được thu thập và truyền đến các node xử lý SLAM nhằm xây

dựng bản đồ môi trường. Đồng thời, ROS2 cũng đảm nhiệm việc trao đổi dữ liệu giữa các thành phần như bộ điều khiển động cơ, hệ thống định vị, thuật toán điều hướng và giao diện giám sát trên máy tính. Thông qua các công cụ như RViz2, người dùng có thể quan sát bản đồ môi trường, dữ liệu quét từ LiDAR và vị trí của robot trong thời gian thực.

Nhờ cấu trúc linh hoạt, khả năng mở rộng cao và hệ sinh thái phong phú, ROS2 đóng vai trò quan trọng trong việc phát triển các hệ thống robot tự hành hiện đại. Việc ứng dụng ROS2 trong đề tài không chỉ giúp đơn giản hóa quá trình phát triển phần mềm mà còn tạo điều kiện thuận lợi cho việc tích hợp các thuật toán tiên tiến như SLAM, điều hướng tự động và tránh vật cản, góp phần nâng cao hiệu quả hoạt động của hệ thống xe tự hành cứu hỏa.

Các thành phần cơ bản của ROS:

“Computation Graph”, gọi là lược đồ tính toán, là một mạng peer-to-peer của ROS trong đó các dữ liệu được xử lý với nhau. Computation Graph cơ bản gồm các thành phần: Nodes, Master, Parameter Server, Messages, Services, Topics, và Bags, tất cả các thành phần này đều cung cấp dữ liệu cho Graph bằng những phương thức khác nhau.

Master: đóng vai trò kết nối các node với nhau. Do đó, master luôn được khởi động đầu tiên bằng câu lệnh `roscore` sau đó thì bạn có thể gọi bất kỳ các node nào trong hệ thống. Sau khi gọi xong, các node có thể kết nối và tương tác với nhau.

Parameter Server: Là một cấu trúc nhiều tham số có thể truy cập trong lúc chạy ROS. Các node sử dụng cấu trúc này nhằm lưu trữ và truy xuất các thông số trong thời gian chạy. Do nó không có hiệu suất cao nên thường dùng với kiểu dữ liệu tĩnh, chẳng hạn như các thông số cấu hình, thời gian hệ thống.

Message: Đây là cấu trúc dữ liệu được các node sử dụng để trao đổi với nhau tương tự như các kiểu dữ liệu `double`, `int` trong các ngôn ngữ lập trình. Các node tương tác với nhau bằng cách `send` và `receive ROS message`.

Topics: là phương pháp giao tiếp trao đổi dữ liệu giữa hai node, nó bao gồm nhiều cấp bậc thông tin mà chúng có thể giao tiếp thông qua ROS message. Hai phương thức trong topic bao gồm `publish` và `subscribe`.

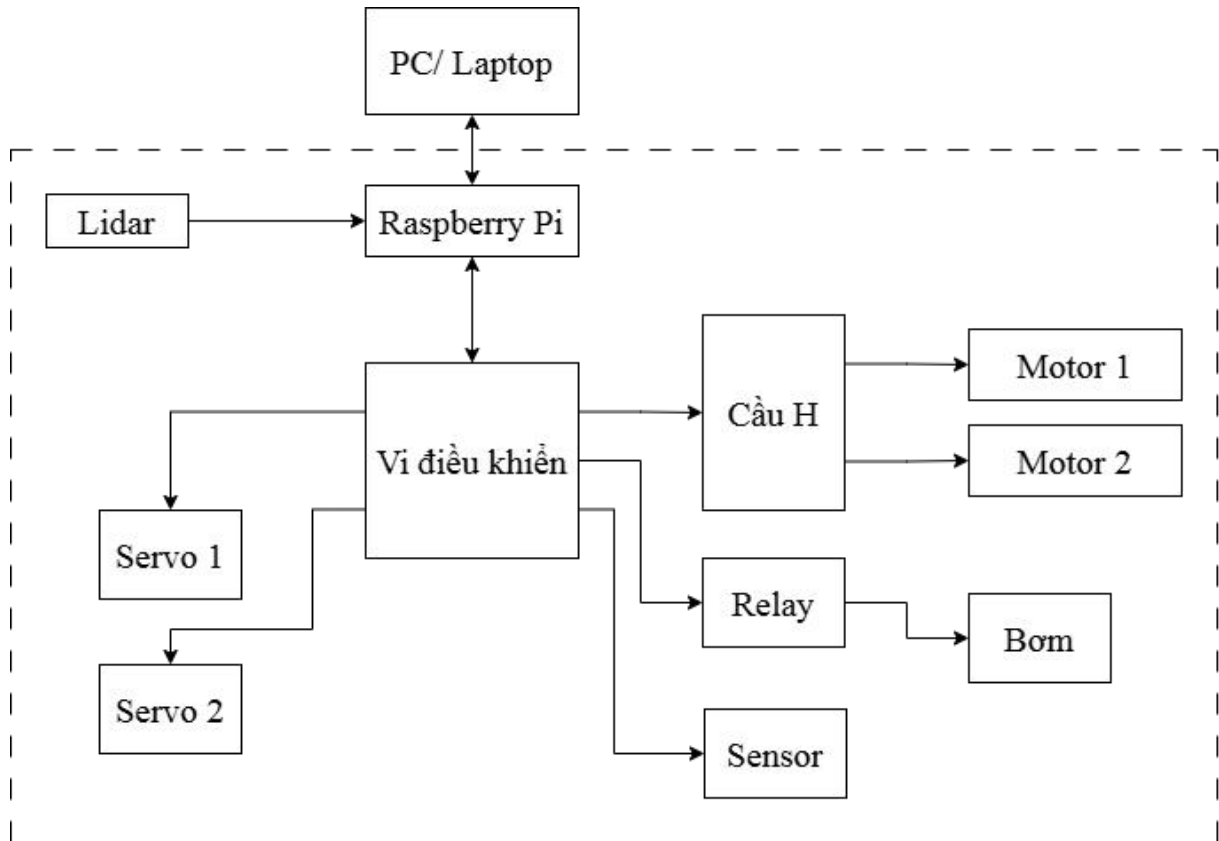
Services: Là một giao tiếp trao đổi dữ liệu/ thông tin giữa hai node thông qua phương thức `request` và `response`. Thường được áp dụng trong trường hợp việc thực hiện một lệnh cần nhiều thời gian xử lý nên dữ liệu tính toán được lưu ở server và sẽ dùng khi cần xử lý.

Bags: là một định dạng tệp trong ROS dùng để lưu trữ dữ liệu message với phần mở rộng là `.bag` có vai trò quan trọng và có thể được xử lý, phân tích bởi các công cụ mô phỏng trong ROS như `Rviz`.

CHƯƠNG 2. THIẾT KẾ PHẦN CỨNG CHO XE TỰ HÀNH CỨU HỎA

2.1. Sơ đồ tổng quát hệ thống

Xe tự hành là một hệ thống được thiết kế gồm có khung cơ khí, các ngoại vi và mạch điều khiển đáp ứng các yêu cầu xử lý cảm biến, tự hành tránh vật cản và điều khiển từ xa.



Hình 6. Sơ đồ khối tổng thể hệ thống xe tự hành cứu hỏa

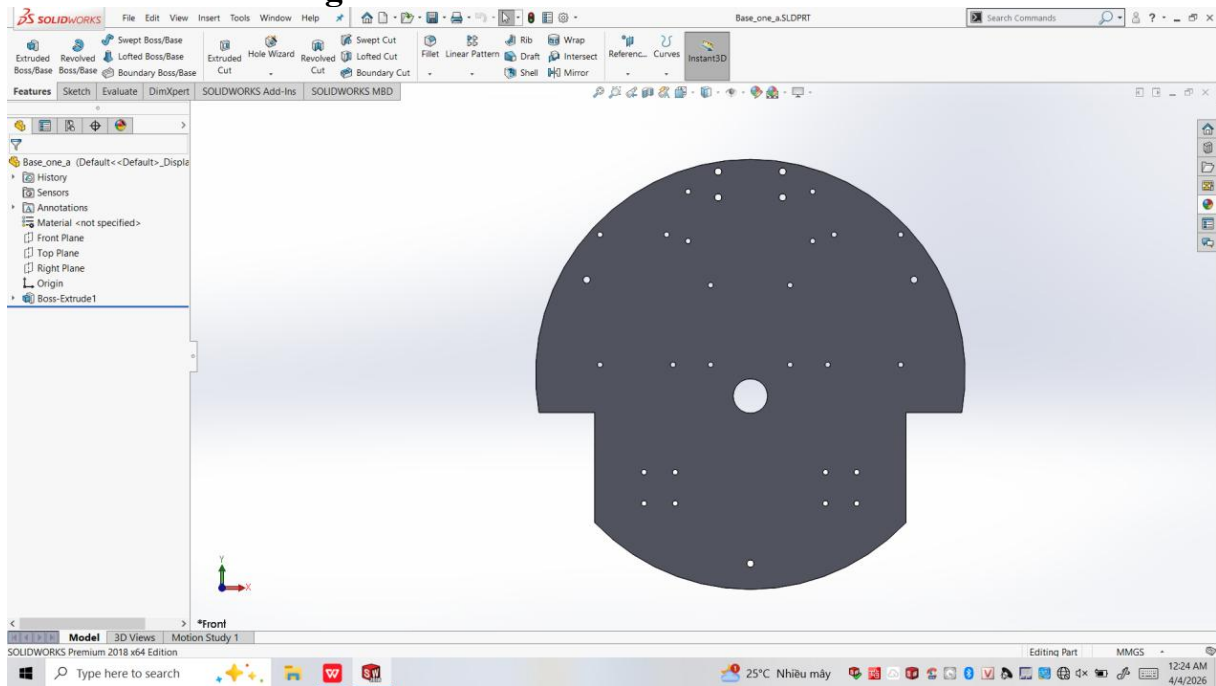
2.2. Lựa chọn thiết bị

Bảng 1. Các thiết bị cho xe tự hành cứu hỏa

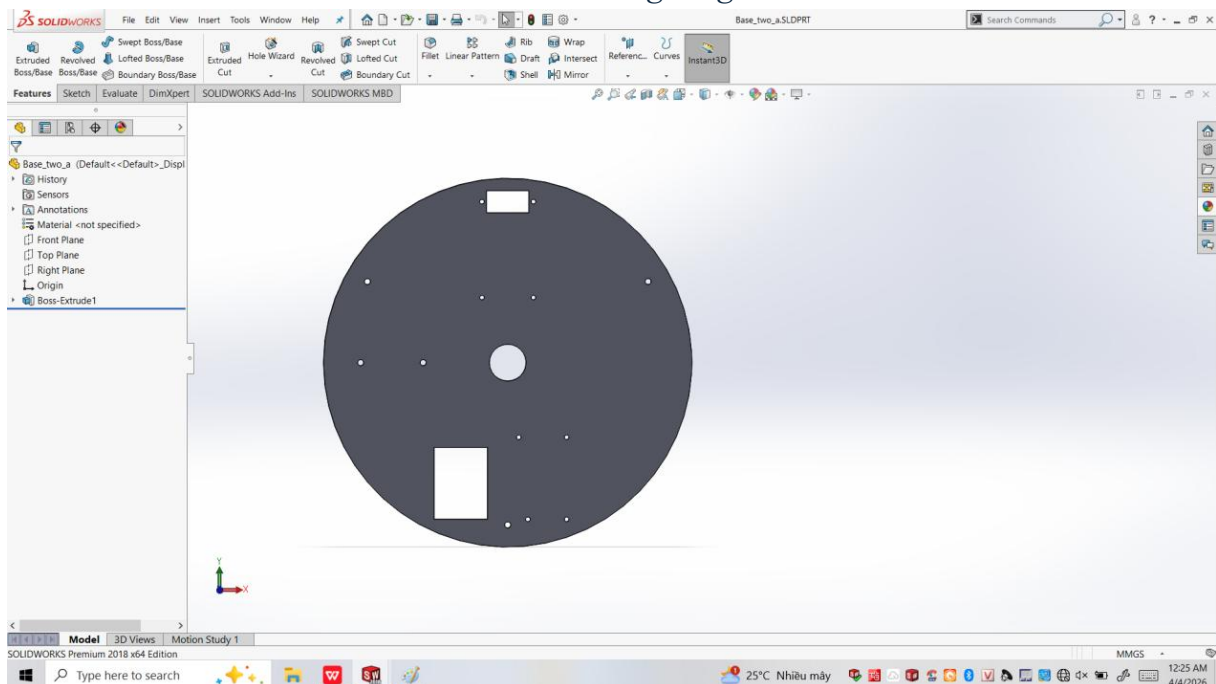
Tên thiết bị	Số lượng
Khung xe	1
Động cơ DC	2
Bánh xe đa hướng	1
Bánh xe Mecanum	2
Raspberry Pi 4	1
ESP32	1
Cảm biến RPLidar A2M12	1
Tay cầm PS3 Bluetooth	1
Mạch cầu H L298N	1

Nguồn điện 12V	2
Module hạ áp	2
Relay	1
Servo	2
Relay	1
Động cơ bơm 12V	1
Cảm biến lửa	1

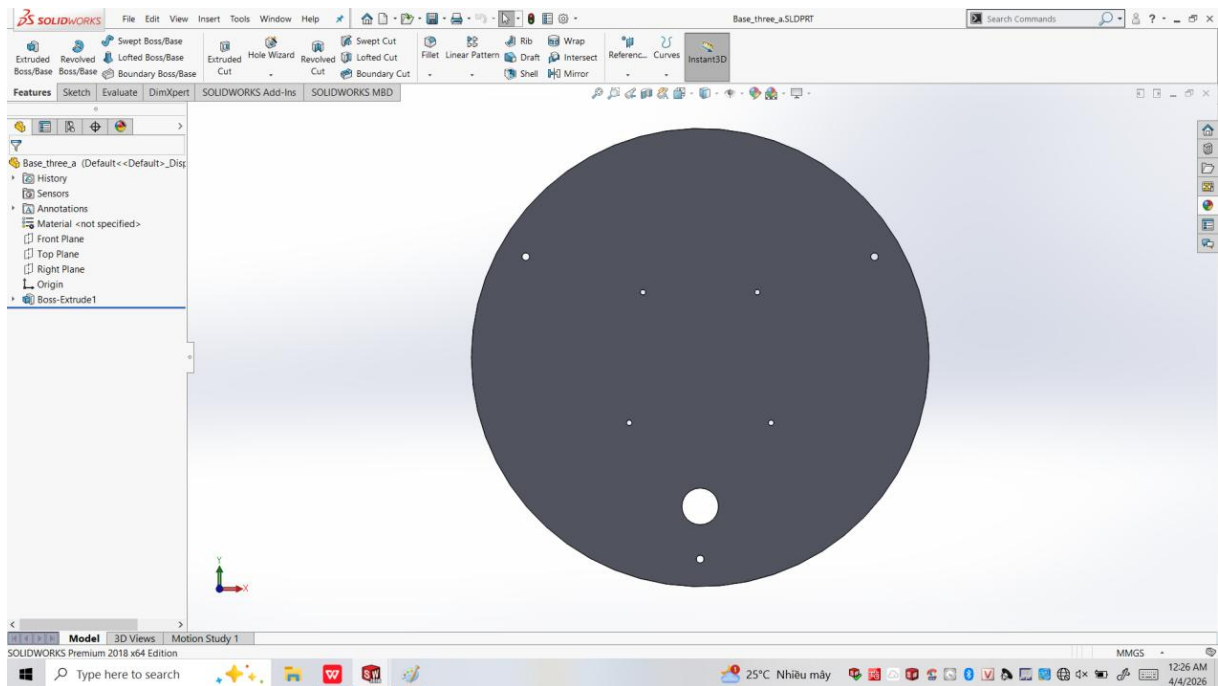
2.2.1. Thiết kế khung xe



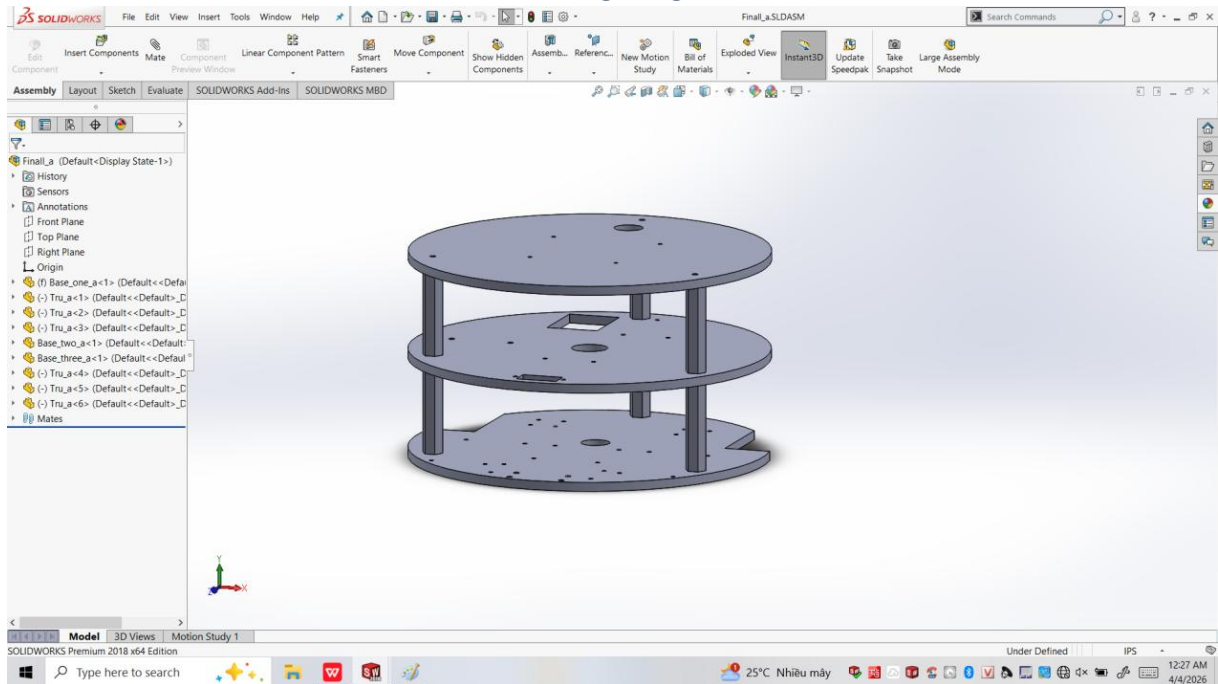
Hình 7. Khung tầng 1 của xe



Hình 8. Khung tầng 2 của xe



Hình 9. Khung tầng 3 của xe



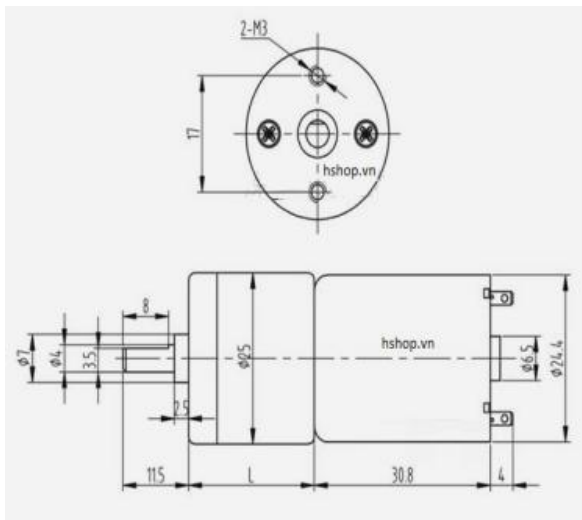
Hình 10. Toàn bộ khung xe sau khi lắp ráp

Xe được thiết kế gồm 3 tầng, mỗi tầng được thiết kế bởi một tấm nhựa Mica có độ dày 3mm. Đặt trung của vật liệu này là có tỉ trọng bằng $\frac{1}{2}$ so với thủy tinh, chịu lực và chịu nhiệt rất tốt, hạn chế tác động của các hóa chất như xăng dầu, bền về mặt thời gian và chi phí vật liệu rẻ hơn so với các vật liệu khác như nhôm, inox,... Khung xe được thiết kế với các lỗ được đục sẵn như hình bên trên.

2.2.2. Lựa chọn động cơ DC

Với tốc độ không yêu cầu di chuyển nhanh, động cơ DC được yêu cầu có hộp số giảm tốc để tăng momen xoắn giúp việc kéo tải của động cơ được dễ dàng.

Từ đó em chọn được động cơ DC JGA25_370:



Hình 11. Động cơ DC JGA25-370

Động cơ DC JGA25-370 DC được chế tạo hoàn toàn từ kim loại cao cấp, không chỉ mang lại độ bền cơ học vượt trội mà còn đảm bảo khả năng hoạt động ổn định, tin cậy trong nhiều môi trường khắc nghiệt, khiến nó trở thành lựa chọn lý tưởng cho các mô hình robot, xe tự hành, thuyền điều khiển từ xa hay các thiết bị cơ khí chính xác. Hộp giảm tốc tích hợp của động cơ cung cấp đa dạng tỉ số truyền, cho phép người dùng dễ dàng cân bằng giữa lực kéo mạnh mẽ và tốc độ vận hành, theo nguyên lý lực kéo càng lớn thì tốc độ càng chậm và ngược lại, nhờ đó tối ưu hóa hiệu suất cho từng ứng dụng cụ thể. Với việc sử dụng lõi dây đồng nguyên chất, lá thép kỹ thuật 407 và nam châm từ tính mạnh, động cơ JGA25-370 DC vượt trội hơn hẳn các loại động cơ giá rẻ trên thị trường (vốn thường dùng lõi nhôm và nam châm yếu), mang đến sức kéo bền bỉ, tuổi thọ cao và hiệu suất làm việc vượt trội trong thời gian dài.

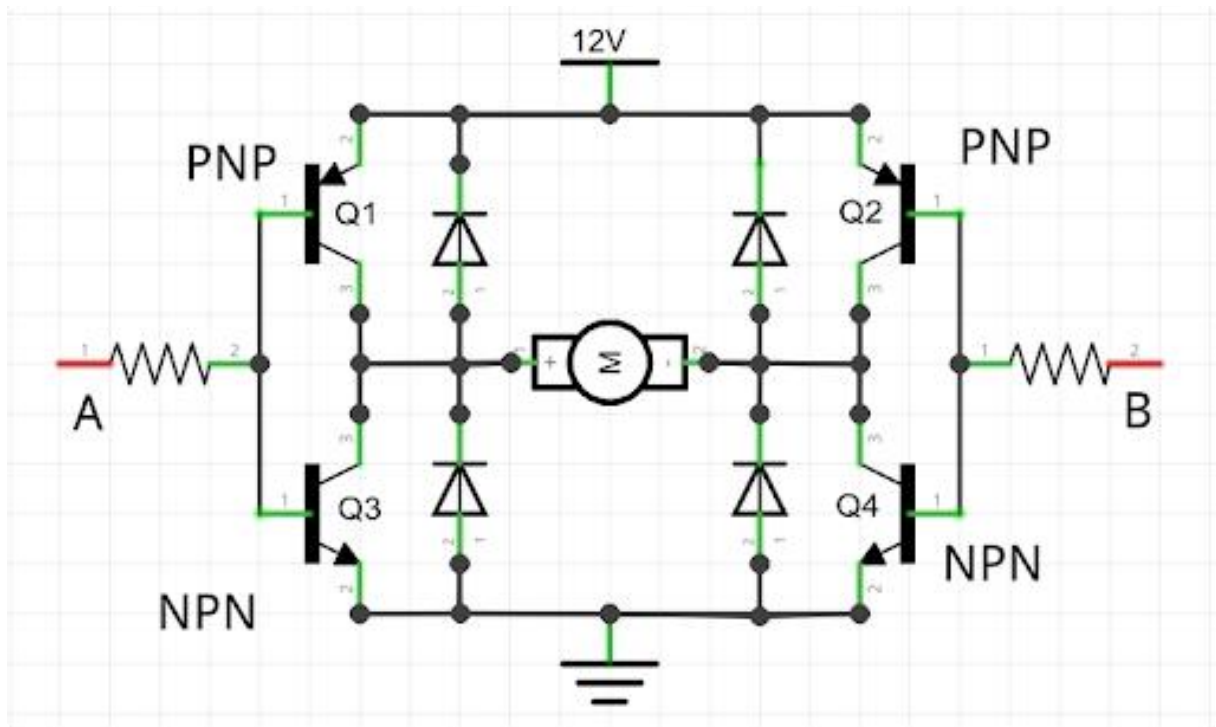
Thông số kỹ thuật:

Bảng 2. Thông số động cơ DC JGA25-370

Model	JGA25-370
Điện áp sử dụng	12VDC
Dòng điện không tải	60mA
Đường kính	25mm
Tỉ số truyền	103
Tốc độ không tải	60 rpm (60 vòng/ phút)
Momen xoắn định mức	2.7 kg.cm
Công suất định mức	3.5 Wat

2.2.3. Lựa chọn mạch cầu H

Lựa chọn cầu H để điều khiển chiều quay, tốc độ động cơ yêu cầu ta cần có một mạch điều khiển chiều quay của động cơ và cường độ dòng điện đi qua động cơ. Mạch cầu H cho phép động cơ quay thuận hoặc quay nghịch thông qua việc thay đổi chiều dòng điện cấp vào động cơ. Ngoài ra, tốc độ động cơ có thể được điều chỉnh bằng phương pháp điều chế độ rộng xung PWM từ vi điều khiển. Việc sử dụng cầu H giúp hệ thống điều khiển động cơ linh hoạt, ổn định và đáp ứng tốt yêu cầu chuyển động của robot.



Hình 12. Sơ đồ nguyên lý cầu H

Mạch cầu H (H-Bridge) là mạch điện tử sử dụng 4 công tắc (transistor hoặc MOSFET) sắp xếp hình chữ H để điều khiển động cơ DC quay thuận, nghịch, dừng hoặc hãm. Nguyên lý dựa trên việc đảo chiều dòng điện qua động cơ bằng cách đóng/cắt các cặp công tắc chéo nhau, cho phép điều khiển tốc độ qua PWM.

Quay thuận: Đóng công tắc 1 và 4 (Q_1, Q_4), dòng điện đi từ dương nguồn qua Q_1 qua động cơ, qua Q_4 xuống âm nguồn. Động cơ quay theo một chiều.

Quay nghịch: Đóng công tắc 2 và 3 (Q_2, Q_3), dòng điện đi từ dương nguồn qua S_2 , qua động cơ theo chiều ngược lại, qua S_3 xuống âm nguồn.

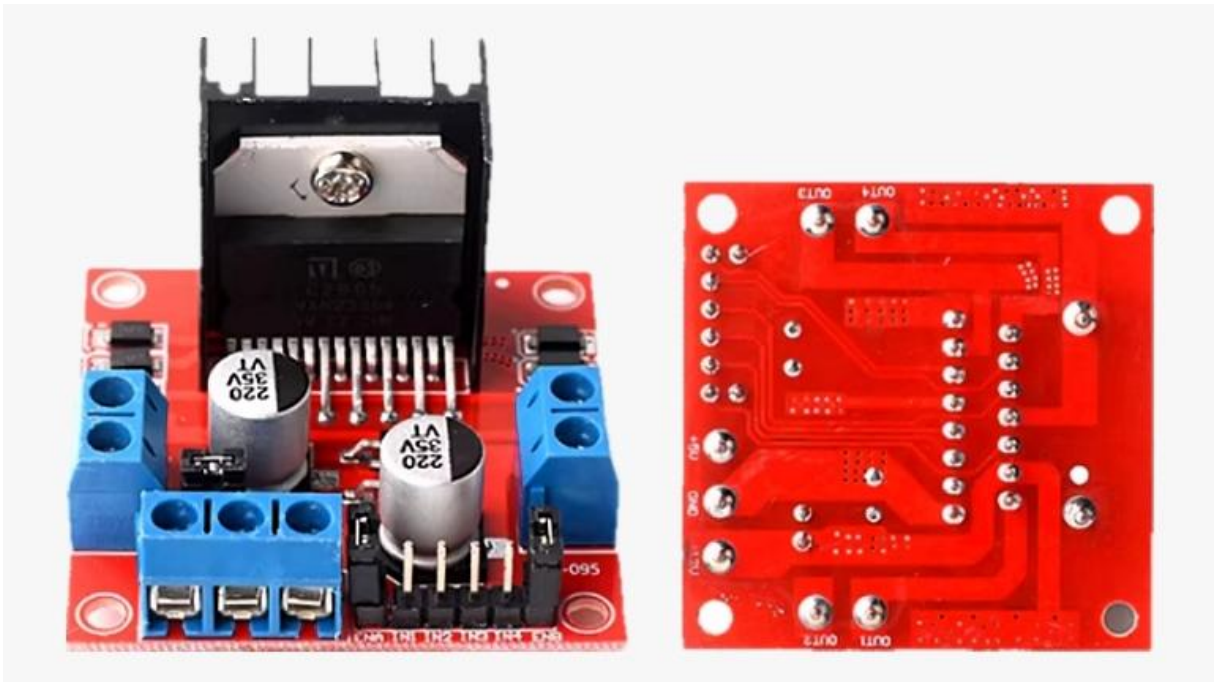
Dừng (Tự do): Mở tất cả các công tắc. Động cơ dừng quay tự do.

Hãm (Brake): Đóng cả Q_1 và Q_3 (hoặc Q_2 và Q_4 để ngắn mạch hai cực động cơ.

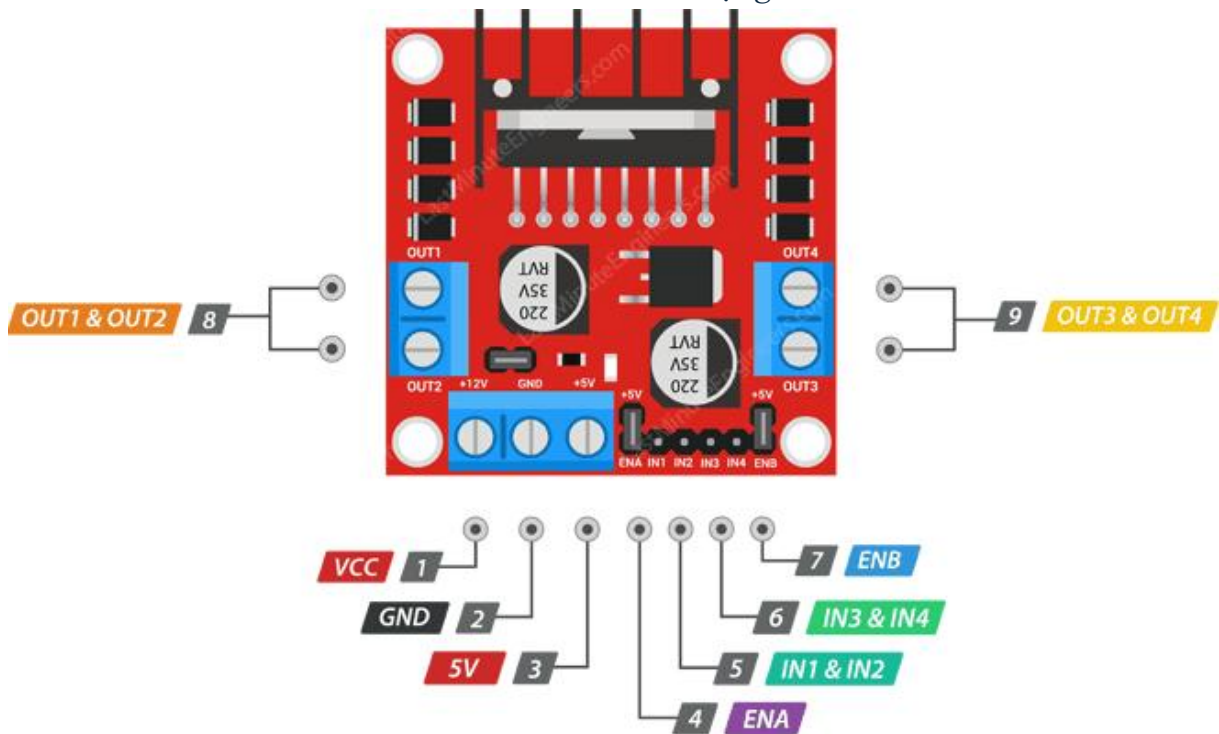
Lựa chọn cầu H thích hợp là một trong những yếu tố quyết định đến chất lượng điều khiển động cơ. Do đó em lựa chọn module L298N, với khả năng điều khiển hai động cơ độc lập, dễ dàng tích hợp và sử dụng.

Module L298N là một mạch điều khiển động cơ một chiều DC cùng lúc. L298N là IC điều khiển cầu kép toàn kỳ có dải điện áp hoạt động rộng, xử lý dòng tải có mức tối đa 3A. Bao gồm điện áp bảo hòa thấp và bảo vệ quá nhiệt. Có cấu tạo từ hai mạch cầu H transistor hoạt động ổn định và hiệu quả cao.

Module điều khiển động cơ L298N được tích hợp 2 mạch cầu H, có khả năng điều khiển 2 động cơ DC và 1 động cơ bước. Dòng ra tối đa 2A và nguồn cấp có thể từ 5 đến 35V. Động cơ A được điều khiển bằng cách cấp các mức điện áp 1 và 0 vào 2 chân IN1 IN2. Tương tự động cơ B được điều khiển bằng cách cấp mức 1, 0 vào IN3 IN4. Đảo các mức 1, 0 này sẽ làm động cơ đảo chiều và có thể điều khiển tốc độ động cơ bằng cách cấp xung trực tiếp vào các chân IN. Chân Enable còn có tác dụng “phanh” gấp động cơ. Khi đưa chân Enable tương ứng với động cơ xuống mức 0.



Hình 13. Module điều khiển động cơ L298N



Hình 14. Chức năng các chân của Module điều khiển động cơ L298N

Thông số kỹ thuật:

Bảng 3. Thông số kỹ thuật của Module điều khiển động cơ L298N

Model	L298N
Sử dụng IC	Công suất L298N
Điện áp tín hiệu	5V/ 0mA-36mA
Điện áp hoạt động động cơ	5V-35V
Dòng điều khiển động cơ	2A/1 mạch cầu H
Nhiệt độ hoạt động	-20 tới +135 độ C

Công suất đầu ra 1 cầu H	25W
Trọng lượng	30g
Kích thước	43*43*27mm

Chức năng các chân kết nối:

Bảng 4. Chức năng các chân của Module điều khiển động cơ L298N

IN1 và IN2	Các chân đầu vào điều khiển hướng quay động cơ A
IN3 và IN4	Các chân đầu vào điều khiển hướng quay động cơ B
ENA	Kích hoạt tín hiệu PWM cho động cơ A
ENB	Kích hoạt tín hiệu PWM cho động cơ B
OUT1 và OUT2	Chân đầu ra cho động cơ A
OUT3 và OUT4	Chân đầu ra cho động cơ B
12V	Đầu vào cấp nguồn 12
5V	Cấp nguồn cho mạch logic bên trong IC L298N
GND	Chân nối đất

2.2.4. Vi điều khiển ESP32

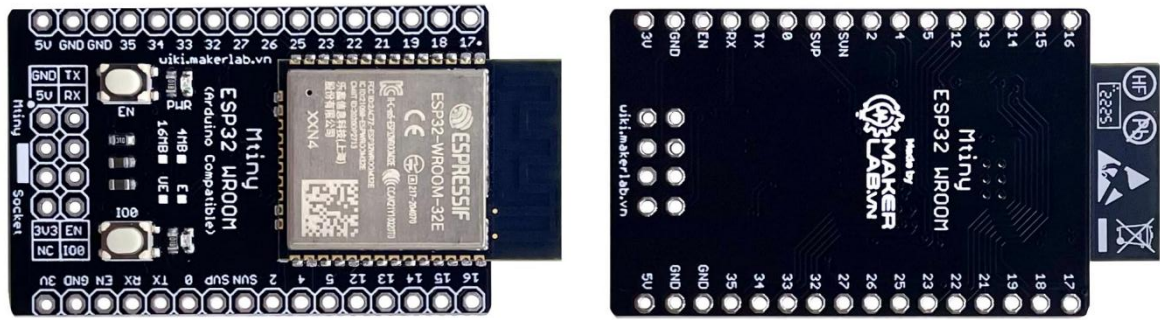
ESP32 là một vi điều khiển nhỏ gọn, mạnh mẽ do Espressif Systems phát triển, được ra mắt lần đầu vào năm 2016. Đây là phiên bản nâng cấp vượt trội so với các dòng vi điều khiển phổ biến như Arduino, đặc biệt nổi bật với khả năng tích hợp WiFi và Bluetooth ngay trên chip.

ESP32 có kích thước nhỏ gọn, phù hợp với nhiều loại mạch và dự án khác nhau, đồng thời vẫn đảm bảo hiệu năng cao nhờ sử dụng vi xử lý dual-core (2 nhân). So với Arduino Nano, ESP32 có tốc độ xử lý nhanh hơn, bộ nhớ lớn hơn và hỗ trợ nhiều chuẩn giao tiếp hiện đại.

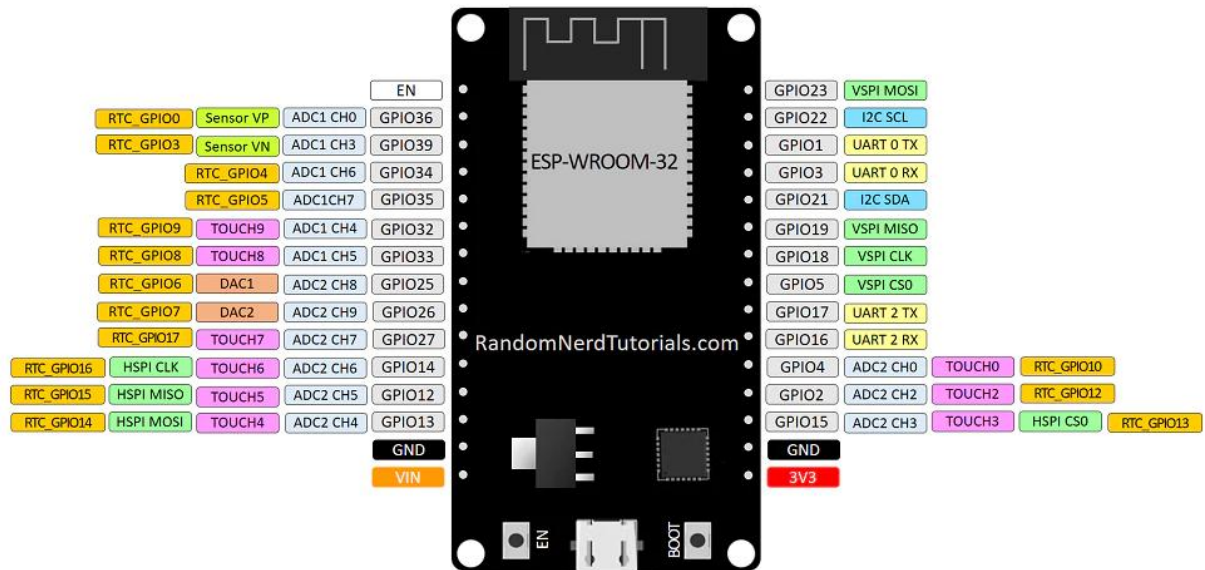
Một ưu điểm nổi bật của ESP32 là khả năng lập trình linh hoạt, có thể sử dụng trực tiếp trên Arduino IDE hoặc các nền tảng khác như MicroPython. Ngoài ra, ESP32 còn hỗ trợ nhiều chân GPIO, ADC, DAC, PWM... giúp dễ dàng kết nối với cảm biến, động cơ và các thiết bị ngoại vi.

Cộng đồng người dùng ESP32 vô cùng lớn mạnh với hàng nghìn thư viện mã nguồn mở, tài liệu hướng dẫn phong phú và các dự án mẫu sẵn có, giúp người mới bắt đầu cũng như kỹ sư giàu kinh nghiệm nhanh chóng phát triển ý tưởng từ nguyên mẫu đến sản phẩm hoàn chỉnh.

Với kích thước nhỏ gọn, tiêu thụ điện năng hợp lý và tích hợp sẵn kết nối không dây, ESP32 được ứng dụng rộng rãi trong các lĩnh vực như IoT (Internet of Things), nhà thông minh, robot tự hành, hệ thống giám sát và điều khiển từ xa.



Hình 15. Vi xử lý ESP32



Hình 16. Sơ đồ chân của ESP32

Bảng 5. Thông số kỹ thuật của ESP32

Vi điều khiển	ESP32
CPU	Dual-core Xtensa LX6
RAM	520 KB SRAM
GPIO	34 chân
ADC	12-bit 15 chân
DAC	8-bit 2 chân
PWM	Tất cả các GPIO
Giao tiếp	UART, SPI, I2C, CAN
Điện áp hoạt động	3V-3.6V (Định mức 3.3V)
Dòng tiêu thụ	80mA (khi Wifi hoạt động)
Nhiệt độ hoạt động	-40 tới +125 độ C
Kích thước	25.5*18mm

Bảng 6. Chức năng các chân của ESP32

VIN	Cấp nguồn 5V
3V3	Nguồn ra 3.3V
GND	Nối đất

GPIO0-GPIO33	Input/Output
GPIO34-GPIO39	Input
GPIO 32, 33, 34, 35, 36, 39	ADC 1
GPIO 0, 2, 12, 13, 14, 15, 25, 26, 27	ADC 2
GPIO 25	DAC 1
GPIO 26	DAC 2
GPIO 1	TX0
GPIO 3	RX0
GPIO 17	TX2
GPIO 16	RX2
GPIO 22	SCL
GPIO 21	SDA

2.2.5. Raspberry Pi 4

Raspberry Pi 4 là một máy tính nhúng (single-board computer) được phát triển bởi Raspberry Pi Foundation. Nó là phiên bản thứ tư trong dòng sản phẩm Raspberry Pi, và được ra mắt vào tháng 6 năm 2019. Raspberry Pi 4 được thiết kế để cung cấp hiệu suất và tính năng cao hơn so với các phiên bản trước đó.

Một số đặc điểm chính của Raspberry Pi 4:

- + Bộ xử lý: Vi xử lý Broadcom BCM2711 lõi tứ 64-bit ARM Cortex A72, chạy ở tốc độ 1.5GHz. So với thế hệ trước (Pi 3B+ với CPU Cortex-A53 1.2GHz), hiệu năng CPU của Pi 4 mạnh hơn đáng kể.

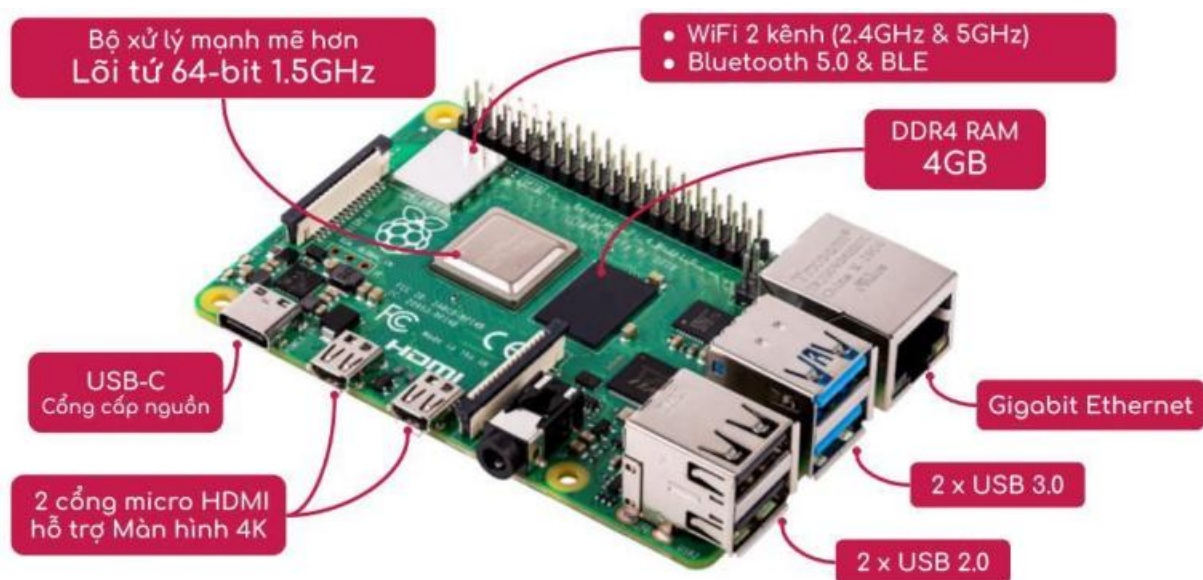
- + Bộ nhớ RAM: Thiết bị có sẵn các phiên bản RAM LPDDR4-2400 với dung lượng 1GB, 2GB, 4GB, và lên đến 8GB. Dung lượng RAM lớn hơn cho phép Raspberry Pi 4 xử lý các tác vụ phức tạp như chỉnh sửa ảnh/video, chạy nhiều ứng dụng cùng lúc hoặc vận hành các mô hình trí tuệ nhân tạo (AI) một cách mượt mà hơn.

- + Kết nối: Cổng mạng 1 cổng Gigabit Ethernet tốc độ cao, cổng USB 2 cổng USB 3.0 và 2 cổng USB 2.0. GPIO chuẩn 40 chân, tương thích ngược với các bo mạch Raspberry Pi trước đây, dùng để kết nối với các cảm biến, động cơ, đèn LED và các thiết bị ngoại vi khác. Kết nối Wi-Fi có hỗ trợ chuẩn 2.4GHz và 5GHz 802.11b/g/n/ac, Bluetooth 5.0 với hỗ trợ BLE.

- + Hiển thị: Pi 4 nổi bật với khả năng xuất hình ảnh chất lượng cao, đầu ra video có 2 cổng micro-HDMI, hỗ trợ xuất tín hiệu ra hai màn hình riêng biệt cùng lúc với độ phân giải lên đến 4K ở tần số quét 60Hz (4Kp60). Cổng giao tiếp màn hình là cổng MIPI DSI 2 làn để kết nối màn hình chính hãng Raspberry Pi.

- + Internet: Cổng Gigabit Ethernet và Wi-Fi dual band (2.4GHz/5GHz), Raspberry Pi 4 đảm bảo kết nối mạng cực kỳ ổn định và tốc độ cao, lý tưởng cho các ứng dụng máy chủ, luồng dữ liệu thời gian thực, hoặc các dự án IoT đòi hỏi băng thông lớn.

- + Hệ điều hành: Raspberry Pi 4 có thể chạy nhiều hệ điều hành, bao gồm Raspbian (hệ điều hành chính thống của Raspberry Pi), Ubuntu, và các phiên bản khác của Linux. Nó cũng có khả năng chạy Windows 10 IoT Core và nhiều hệ điều hành nhúng khác.



Hình 17. Máy tính nhúng Raspberry Pi 4

Thông số kỹ thuật:

Bảng 7. Thông số kỹ thuật Raspberry Pi 4

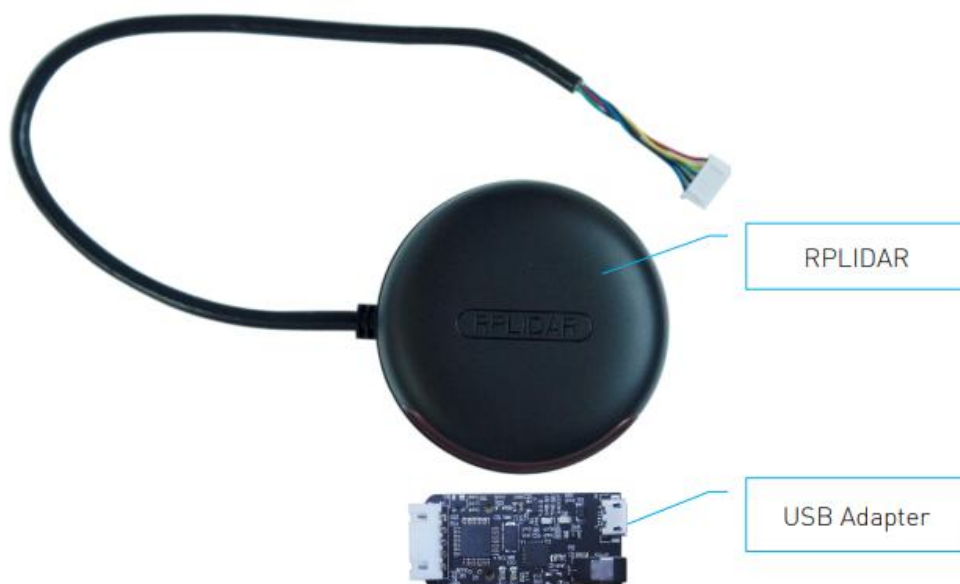
Bộ xử lý	ARM Cortex-A76, Quad-core 64-bit.
Tốc độ xung	2.4 GHz
Bộ nhớ	4 GB LPDDR4X RAM.
Kết nối mạng	Ethernet, Wi-Fi, Bluetooth.
Cổng HDMI	Cổng xuất hình ảnh và âm thanh ra màn hình hoặc TV
USB Ports	Các cổng USB để kết nối chuột, bàn phím, cảm biến, ổ cứng, camera,...
Khe cắm thẻ nhớ	Dùng để lưu hệ điều hành và dữ liệu.
Cổng nguồn	Cổng cấp nguồn cho thiết bị USB-C
GPIO	40 chân GPIO, Hỗ trợ I2C, SPI, UART, PWM, và các giao tiếp khác
Hệ điều hành hỗ trợ	Raspbian, Ubuntu, Windows 10 IoT Core, Nhiều hệ điều hành Linux, ...
Kích thước	85.6mm x 56.5mm
Điện áp đầu vào	5V
Dòng điện định mức	3A

2.2.6. Cảm biến RPLidar A2M12

Cảm biến RPLIDAR A2M12 là một giải pháp Lidar 2D 360 độ, chi phí thấp do SLAMTEC phát triển, hoạt động dựa trên nguyên lý đo khoảng cách bằng laser tam giác (laser triangulation). Thiết bị này quét xung quanh để thu thập dữ liệu điểm (point cloud) trong phạm vi bán kính lên tới 12 mét (với vật thể trắng) và có thể đạt tới 16 mét nhờ tối ưu thuật toán, với tốc độ lấy mẫu lên đến 16.000 lần mỗi giây. Nhờ sử dụng động cơ không chổi than (brushless motor) và công nghệ truyền dữ liệu, điện

năng không tiếp xúc OPTMAG, A2M12 vừa vận hành êm ái vừa khắc phục được nhược điểm mài mòn cơ học.

Với độ phân giải góc cao 0.225° và tần số quét có thể điều chỉnh từ 5-15Hz (10Hz tiêu chuẩn), cảm biến cung cấp khả năng xây dựng bản đồ và xác định vị trí trong nhà (indoor) một cách chi tiết. RPLIDAR A2M12 sử dụng tia laser hồng ngoại công suất thấp, đạt chuẩn an toàn mắt Class 1 của IEC-60825, và có thiết kế siêu mỏng chỉ 4cm, rất phù hợp để lắp đặt trên các loại robot dịch vụ, robot hút bụi, cũng như trong các hệ thống SLAM, tương tác đa điểm, và dẫn đường tự động.

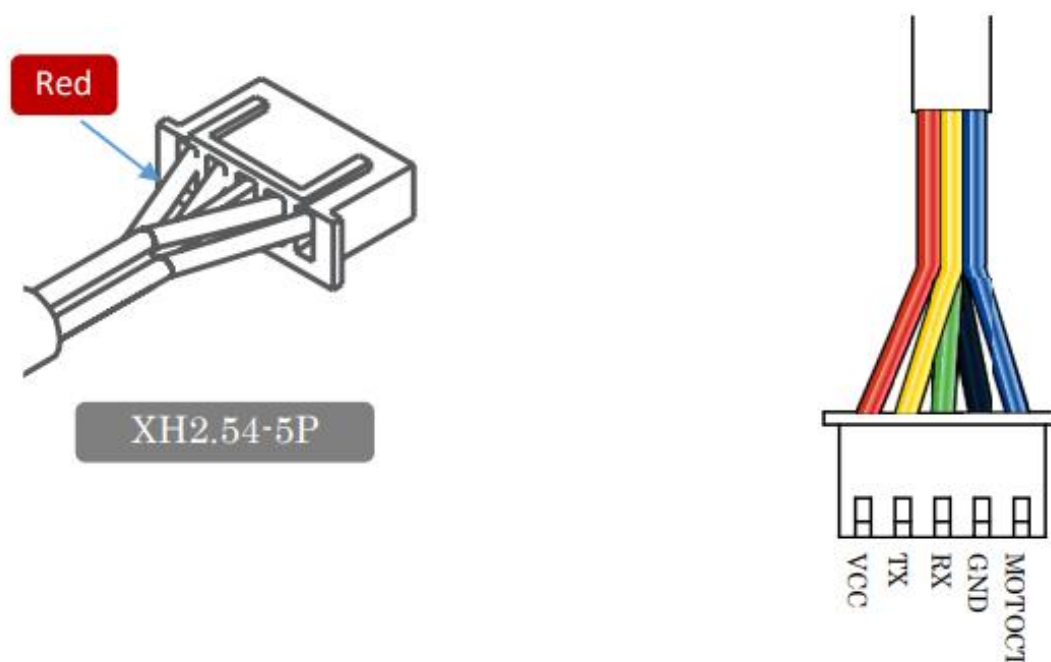


Hình 18. Cảm biến RPLiDAR A2M12

Thông số kỹ thuật:

Bảng 8. Thông số kỹ thuật cảm biến LiDAR A2M12

Model	A2M12
Góc quét	360 độ
Tần số lấy mẫu	16000 lần/ giây (16kHz)
Tần số quét	5Hz- 15Hz
Kích thước	76mm x 76mm x 41mm
Trọng lượng	190 g
Độ chính xác	0.225 độ
Giao tiếp	UART
Tốc độ Baud	256000 bps
Nguồn điện	5VDC
Dòng điện hoạt động	450-600mA
Khoảng cách đo	0.2-12m
Nhiệt độ hoạt động	0°C đến 40°C
Bước sóng laser	785nm



Hình 19. Sơ đồ chân cảm biến RPLiDAR A2M12

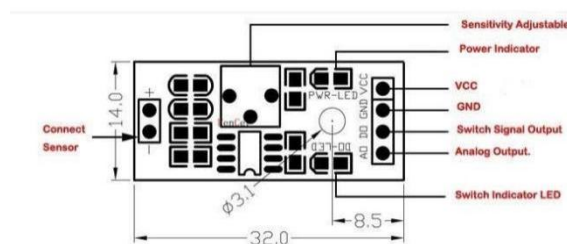
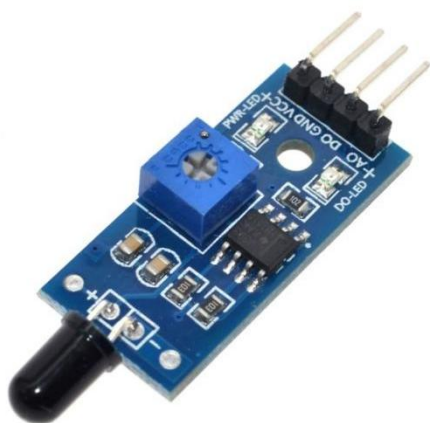
Bảng 9. Chức năng các chân của cảm biến RPLiDAR A2M12

Tên chân	Ý nghĩa
GND	Chân nối đất
RX	Nhận dữ liệu
TX	Gửi dữ liệu
VCC	Nguồn cấp 5VDC
MOTOCT	Điều khiển động cơ

Bên cạnh đó, cảm biến hỗ trợ giao tiếp UART giúp dễ dàng kết nối với các vi điều khiển và máy tính nhúng như Raspberry Pi hoặc Jetson Nano. Dữ liệu quét từ RPLIDAR có thể được sử dụng để phát hiện vật cản, lập bản đồ môi trường và hỗ trợ điều hướng tự động cho robot. Nhờ tốc độ quét nhanh và độ chính xác cao, cảm biến giúp robot phản ứng kịp thời với các thay đổi trong môi trường hoạt động. Ngoài ra, thiết bị còn được hỗ trợ bởi nhiều thư viện và nền tảng mã nguồn mở như ROS, giúp việc lập trình và tích hợp hệ thống trở nên thuận tiện hơn. Với các ưu điểm về hiệu năng, kích thước và độ ổn định, RPLIDAR A2M12 là lựa chọn phù hợp cho các hệ thống robot tự hành trong nhà.

2.2.7. Cảm biến lửa

Cảm biến lửa hay còn gọi là cảm biến bức xạ hoạt động như một "mắt" thụ động, chỉ phản ứng khi có bức xạ hồng ngoại mạnh từ bên ngoài như ngọn lửa, tia lửa hàn trong dải bước sóng của nó thường từ 760nm - 1100nm. Loại cảm biến này thường được tích hợp trong các hệ thống báo cháy tự động tại nhà xưởng, kho xăng dầu hoặc trạm năng lượng. Khi phát hiện bức xạ hồng ngoại đặc trưng của ngọn lửa (có tần số nhấp nháy khoảng 1–10 Hz), nó sẽ kích hoạt còi báo động hoặc hệ thống chữa cháy. Nhờ hoạt động thụ động, cảm biến không phát ra tín hiệu nào, do đó tiết kiệm năng lượng và tránh gây nhiễu trong môi trường có nhiều nguồn nhiệt ổn định.



Hình 20. Cảm biến lửa

Thông số kỹ thuật:

- + Nguồn cấp: 3.3V – 5VDC
- + Dòng tiêu thụ: 15mA
- + Tín hiệu ra: Digital 3.3 – 5VDC tùy nguồn cấp hoặc Analog.
- + Khoảng cách: 80cm
- + Góc quét: 60 độ
- + Kích thước: 3.2 x 1.4 cm

2.2.8. Servo MG90S

Servo MG90S còn gọi là Servo 9g là loại servo mini rất phổ biến trong giới làm robot, máy bay mô hình, xe tự hành. Điểm mạnh của nó là kích thước siêu nhỏ gọn, trọng lượng rất nhẹ, giá thành rẻ và dễ điều khiển.

Vì nhỏ gọn, MG90S phù hợp để gắn vào những robot cỡ nhỏ, dùng để quay cảm biến, đóng mở kẹp, hoặc điều khiển hướng phun của vòi nước.



Thông số kỹ thuật:

- + Điện áp hoạt động: 4.8V - 6VDC
- + Mô-men xoắn (Lực kéo): 1.8kg/cm
- + Tốc độ hoạt động: 0.10s/60°
- + Góc quay: Thường là 0 - 180°
- + Chất liệu bánh răng: Kim loại
- + Trọng lượng: ~ 13.4g
- + Kích thước: 22.8 × 12.2 × 28.5mm
- + Nhiệt độ hoạt động: 0°C ~ +55°C
- + Chiều dài cáp: 25 cm

Hình 21. Động cơ servo MG90S

2.2.9. Động cơ bơm nước

Động cơ bơm nước 12V tự mồi là dòng máy bơm mini sử dụng điện áp 12V DC, nổi bật với khả năng tự hút nước từ nguồn chứa mà không cần phải mồi nước bằng tay. Phổ biến nhất là các loại sử dụng motor 385, phù hợp cho DIY, hệ thống tưới cây nhỏ giọt, bể cá, hoặc làm mát bằng phun sương.



Thông số kỹ thuật:

- + Điện áp hoạt động: 12VDC
- + Dòng điện định mức: 0.26A
- + Trọng lượng: 80g
- + Lưu lượng bơm: 1 lít/ phút
- + Chiều cao hút và đẩy: 1-2m
- + Đường kính đầu nước: 8mm
- + Kích thước động cơ: 27 × 60–75 mm
- + Độ ồn: < 60–65 dB
- + Loại bơm: bơm màng tự hút (self-priming diaphragm pump)

Hình 22. Động cơ bơm nước mini 12V

2.2.9. Nguồn

Hệ thống sử dụng 2 nguồn chính cấp nguồn cho mạch động lực và mạch điều khiển đều sử dụng pin 18650 mắc theo cấu trúc 3s để đạt 12V sử dụng mạch hạ áp LM2596 và XL4015

Pin 18650 là loại pin Lithium-ion (Li-ion) hình trụ, được sử dụng rất phổ biến trong các thiết bị điện tử, dụng cụ điện cầm tay, xe đạp điện, robot, và các hệ thống lưu trữ năng lượng. Tên gọi "18650" thể hiện kích thước vật lý của pin: đường kính 18mm và chiều dài 65mm.

Pin 18650 có khả năng sạc lại nhiều lần (từ 300 đến 500 chu kỳ), giúp tiết kiệm chi phí và thân thiện với môi trường so với pin dùng một lần. Với mật độ năng lượng cao, kích thước nhỏ gọn, pin 18650 là lựa chọn lý tưởng cho các hệ thống robot di động yêu cầu nguồn cung cấp ổn định và thời gian hoạt động lâu dài.



Thông số kỹ thuật:

- + Kích thước: 18 x 65mm
- + Điện áp định danh: 3.7V
- + Dung lượng 20000mAh
- + Trọng lượng : 45g
- + Nhiệt độ hoạt động: -20°C- 60°C
- + Dòng xả liên tục tối đa: 10A
- + Hóa học pin: Li-ion NCR
- + Ứng dụng: xe tự hành

Hình 23. Pin 18650

LM2596 là một vi mạch chuyển đổi nguồn hạ áp (buck converter) do Texas Instruments sản xuất, thuộc dòng sản phẩm SIMPLE SWITCHER® nổi tiếng với thiết kế đơn giản và hiệu quả. Đây là giải pháp phổ biến cho các ứng dụng cần hạ điện áp từ nguồn cao xuống mức thấp hơn với dòng tải lên đến 3A.

IC LM2596 hoạt động ở chế độ đóng cắt với tần số cố định 150kHz, cho phép sử dụng các linh kiện lọc có kích thước nhỏ hơn so với các bộ chuyển đổi tần số thấp.

Nhờ thiết kế này, LM2596 đạt hiệu suất chuyển đổi cao (lên đến 80-90%) và tỏa nhiệt thấp, rất phù hợp cho các mạch nguồn hạ áp trong thiết bị điện tử công suất vừa và nhỏ. Điểm nổi bật của LM2596 là chỉ cần rất ít linh kiện bên ngoài (tối thiểu 4 linh kiện) để tạo thành một bộ nguồn hoàn chỉnh, giúp tiết kiệm không gian và chi phí.

LM2596 có điện áp đầu ra cố định 5V và phiên bản điều chỉnh được (ADJ) cho phép người dùng thiết lập điện áp đầu ra tùy ý trong khoảng 1.2V đến 37V.



Thông số kỹ thuật:

- + Điện áp đầu vào: 4.5-40V
- + Điện áp đầu ra: 1.23-37V
- + Dòng ra tối đa: 3A
- + Hiệu suất chuyển đổi: ± 4
- + Bảo vệ quá dòng: 3.6-6.9A
- + Bảo vệ quá nhiệt: 125°C

Hình 24. Mạch hạ áp LM2596

XL4015 là một vi mạch chuyển đổi nguồn hạ áp (buck converter) do hãng XLSEMI sản xuất, hoạt động ở chế độ PWM (Pulse Width Modulation) với tần số chuyển mạch cố định 180kHz. Đây là giải pháp lý tưởng cho các ứng dụng cần hạ điện áp từ nguồn cao xuống mức thấp hơn, có khả năng cung cấp dòng tải lên đến 5A với hiệu suất chuyển đổi cao.

Điểm nổi bật của XL4015 so với các dòng IC hạ áp phổ biến khác (như LM2596) là khả năng chịu dòng cao hơn (5A so với 3A) và hiệu suất chuyển đổi tối ưu hơn trong cùng điều kiện hoạt động. Vi mạch được thiết kế với tối ưu các linh kiện bên ngoài, bao gồm bù tần số nội và bộ tạo dao động tần số cố định, giúp mạch thiết kế nhỏ gọn và dễ sử dụng.



Thông số kỹ thuật:

- + Điện áp đầu vào: 4V – 38V DC
- + Điện áp đầu ra: 1.25V – 36V DC
- + Dòng điện đầu ra tối đa: 5A
- + Công suất tối đa: 75W
- + Tần số chuyển mạch: 180kHz
- + Hiệu suất chuyển đổi: 96%
- + Ngưỡng giới hạn dòng: 7A
- + Nhiệt độ hoạt động: -40°C đến +85°C

Hình 25. Mạch hạ áp XL4015

2.2.10. Tay cầm PS3 Bluetooth

Tay cầm PlayStation 3 (DualShock 3) là thiết bị điều khiển không dây chuẩn của Sony dành cho hệ máy console PS3. Sản phẩm nổi bật với công nghệ Bluetooth 2.0 cho phép kết nối không dây ổn định trong bán kính lên đến 8-10 mét, cùng hệ thống phản hồi rung (vibration) giúp người chơi cảm nhận rõ từng tác động trong game. Tay cầm còn được trang bị cảm biến chuyển động SIXAXIS, cho phép nhận diện các thao tác nghiêng, xoay mà không cần nút bấm.

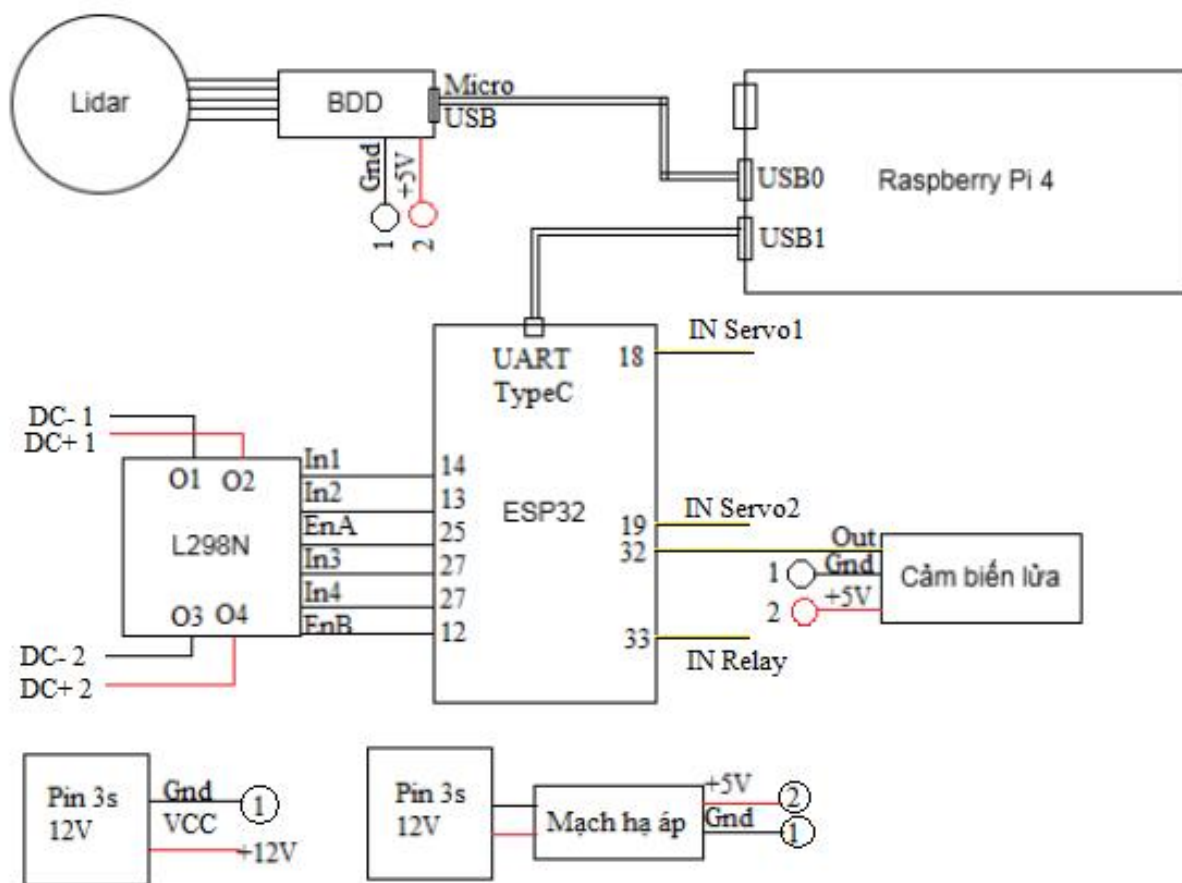


Thông số kỹ thuật:

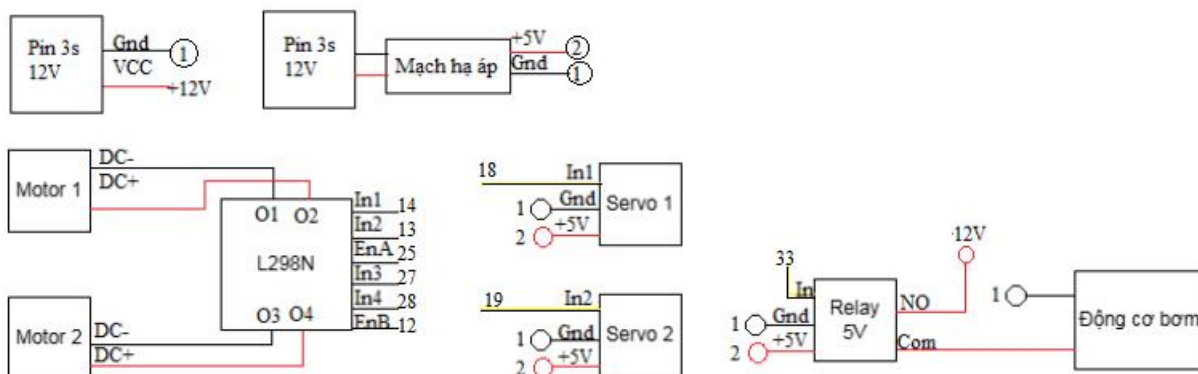
- + Model: DualShock 3 / Sixaxis (CECH-ZC2E).
- + Kết nối: Bluetooth 2.0, tần số 2.4GHz.
- + Nguồn: Pin sạc Lithium-ion tích hợp.
- + Thời lượng pin: 10 giờ liên tục.
- + Tương thích: PlayStation 3, PC (Windows/Linux cần driver), ESP32, các hệ thống nhúng khác.
- + Số lượng kết nối tối đa: 7 tay cầm cùng lúc trên PS3.
- + Tính năng đặc biệt: Cảm biến chuyển động SIXAXIS.

Hình 26. Tay cầm PS3 Bluetooth

2.2.11. Sơ đồ đấu nối



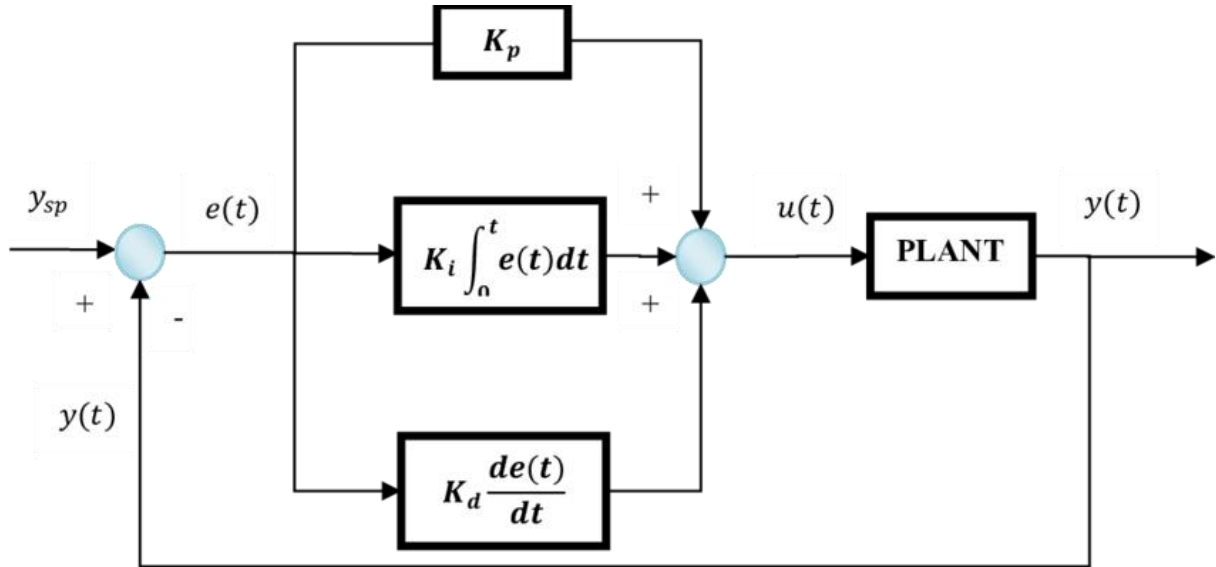
Hình 27. Sơ đồ nguyên lý mạch điều khiển



Hình 28. Sơ đồ nguyên lý mạch công suất

CHƯƠNG 3: XÂY DỰNG THUẬT TOÁN ĐIỀU KHIỂN CHUYỂN ĐỘNG CHO ROBOT

3.1. Điều khiển động cơ DC sử dụng thuật toán PID



Hình 29. Sơ đồ PID

PID được viết tắt bởi cụm từ Proportional Integral Derivative, có nghĩa là 1 cơ chế phản hồi các vòng điều khiển, chúng được ứng dụng rộng rãi trong hệ thống điều khiển công nghiệp hiện đại.

Bộ điều khiển này sử dụng nhiều trong những hệ thống điều khiển vòng kín có tín hiệu phản hồi. Nhiệm vụ của PID giúp tính toán giá trị sai số là hiệu số giữa giá trị đo thông số biến đổi với giá trị đặt mong muốn.

Bộ thiết bị làm giảm tối đa sai số bằng cách điều chỉnh giá trị điều khiển đầu vào. Để đạt được hiệu quả mong muốn bởi thông số của PID cần phải thực hiện điều chỉnh theo tính chất hệ thống. Việc điều khiển sẽ giống nhau, còn thông số được phụ thuộc vào chính đặc thù của hệ thống đó.

PID gồm có 3 khâu hiệu chỉnh gồm khâu tỉ lệ (P), khâu tích phân (I) và khâu vi phân (D)

Ngõ ra của bộ điều khiển PID chính là tổng của ngõ ra ba khâu này

$$Output = P + I + D \quad (1)$$

Khâu P (Tỉ lệ - Proportional) là tích giữa sai số hiện tại với hằng số độ lợi K_p :

$$P = K_p \times \text{Sai số} \quad (2)$$

+ Chức năng: Tạo ra phản hồi tỉ lệ thuận với sai số hiện tại. Sai số càng lớn, P càng lớn.

+ Tác dụng: Giúp hệ thống phản ứng nhanh với sai số, kéo về gần setpoint.

+ Hạn chế: Không thể triệt tiêu hoàn toàn sai số (luôn có sai số dư - offset), dễ gây vọt lố (overshoot) nếu K_p quá lớn.

+ Ví dụ xe tự hành: Sai số hướng (góc lệch) là 10° , $K_p = 0.5 \rightarrow P = 5^\circ$ bánh lái.

Khâu I (Tích phân - Integral) là tích giữa tổng sai số tức thời theo thời gian với độ lợi K_i :

$$I = K_i \times \text{Tích lũy sai số theo thời gian} \quad (3)$$

- + Chức năng: Tổng hợp (tích phân) tất cả các sai số trong quá khứ.
- + Tác dụng: Triệt tiêu hoàn toàn sai số dư, đưa hệ thống về chính xác setpoint.
- + Hạn chế: Gây hiện tượng "integral windup" (tích phân quá mức) nếu sai số kéo dài, dẫn đến overshoot lớn. Cần giới hạn giá trị I.
- + Ví dụ xe tự hành: Xe bị lệch 2° kéo dài 5 giây $\rightarrow$ I tích lũy dần, giúp xe thẳng dần về đúng hướng.

Khâu D (Vi phân - Derivative) là tích giữa độ dốc sai số theo thời gian với độ lợi vi phân :

$$D = K_d \times (\text{Tốc độ thay đổi của sai số}) \quad (4)$$

- + Chức năng: Dự đoán xu hướng thay đổi của sai số trong tương lai, dựa trên đạo hàm (tốc độ thay đổi).
- + Tác dụng: Giảm overshoot, chống dao động, làm hệ thống ổn định nhanh.
- + Hạn chế: Rất nhạy với nhiễu (vì đạo hàm làm khuếch đại nhiễu cao tần). Thường bỏ qua D trong hệ thống có nhiễu nhiều.
- + Ví dụ xe tự hành: Sai số đang giảm nhanh (xe quay về đúng hướng) $\rightarrow$ D âm, giảm bớt lực đánh lái để không quá đà.

Qua quá trình thử nghiệm em đã chọn ra thông số PID ổn định như sau:

$$K_p = 1.5, K_i = 0.05, K_d = 1.8$$

3.2. Xác định vị trí và định hướng robot

Robot được thiết kế theo mô hình dẫn động vi sai (Differential Drive) với hai bánh chủ động độc lập. Việc điều khiển hướng chuyển động được thực hiện bằng cách thay đổi tốc độ quay tương đối giữa hai động cơ.

Do hệ thống không sử dụng encoder nên không thể xác định chính xác quãng đường di chuyển của từng bánh xe thông qua phương pháp odometry truyền thống. Thay vào đó, đề tài sử dụng dữ liệu từ cảm biến LiDAR kết hợp với thuật toán Laser Scan Matcher và SLAM Toolbox trong ROS2 để xác định vị trí và hướng của robot.

Giả sử trạng thái của robot trong mặt phẳng được biểu diễn bởi:

$$P = (x, y, \theta) \quad (5)$$

Trong đó:

- + (x) là tọa độ theo trục X.
- + (y) là tọa độ theo trục Y.
- + (θ) là góc định hướng của robot.

Cảm biến LiDAR liên tục quét môi trường xung quanh với góc quét 360° . Từ tập dữ liệu khoảng cách thu được, thuật toán Scan Matching sẽ so sánh các lần quét liên tiếp để xác định sự thay đổi vị trí và hướng của robot trong môi trường.

Khi robot di chuyển, thuật toán SLAM Toolbox thực hiện hai nhiệm vụ chính:

- + Xây dựng bản đồ môi trường từ dữ liệu LiDAR.
- + Ước lượng vị trí hiện tại của robot trên bản đồ.

Thông qua việc tối ưu sự tương đồng giữa các lần quét LiDAR liên tiếp, hệ thống có thể xác định được tọa độ và góc quay của robot mà không cần sử dụng encoder.

Đối với hệ dẫn động vi sai, mối quan hệ động học của robot được biểu diễn bởi:

$$v = \frac{v_r + v_l}{2} \quad (6)$$

$$\omega = \frac{v_r + v_l}{L} \quad (7)$$

Trong đó:

- + (v): vận tốc tuyến tính của robot.
- + (ω): vận tốc góc của robot.
- + (v_r): vận tốc bánh phải.
- + (v_l): vận tốc bánh trái.
- + (L): khoảng cách giữa hai bánh xe.

Hai động cơ DC giảm tốc có tốc độ định mức 60 vòng/phút (60 RPM), tương đương: $n = 60$; $RPM = 1$; vòng/s.

Nếu bánh xe có bán kính (R), vận tốc tuyến tính lý thuyết của bánh xe được tính theo:

$$v = 2 \times \pi \times R \quad (8)$$

Tuy nhiên trong thực tế vận tốc còn phụ thuộc vào tải trọng, điện áp nguồn, ma sát bề mặt và thuật toán điều khiển động cơ.

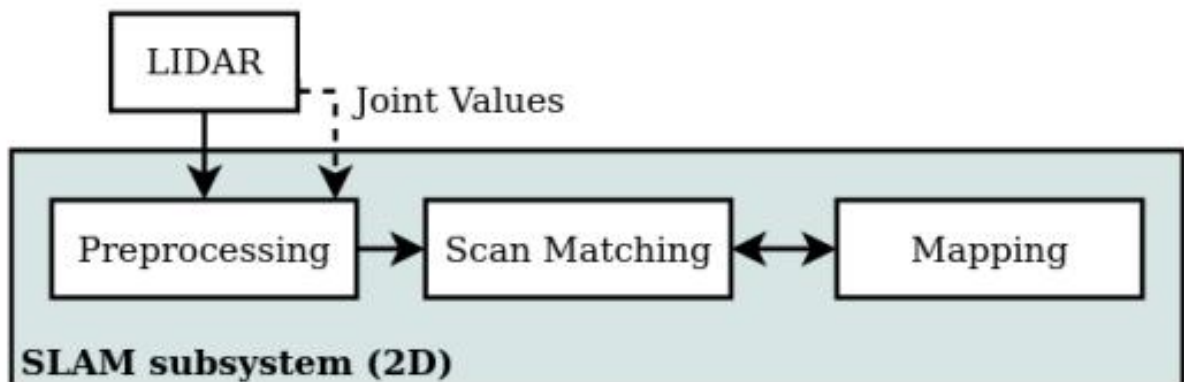
Nhờ sử dụng LiDAR và thuật toán SLAM, robot có khả năng xác định vị trí, xây dựng bản đồ và điều hướng trong môi trường mà không cần trang bị encoder.

3.3. Thuật toán xây dựng bản đồ bằng SLAM Toolbox

Quá trình xây dựng bản đồ môi trường được thực hiện bằng gói phần mềm SLAM Toolbox trên nền tảng ROS2. SLAM Toolbox là một giải pháp mã nguồn mở cho phép robot thực hiện đồng thời hai nhiệm vụ là xây dựng bản đồ môi trường (Mapping) và xác định vị trí của robot trên bản đồ (Localization).

SLAM Toolbox sử dụng dữ liệu từ cảm biến LiDAR để tạo ra bản đồ lưới chiếm dụng (Occupancy Grid Map) của môi trường hoạt động. Thông qua việc so sánh các lần quét LiDAR liên tiếp, thuật toán có thể xác định sự thay đổi vị trí và hướng của robot, từ đó cập nhật bản đồ theo thời gian thực.

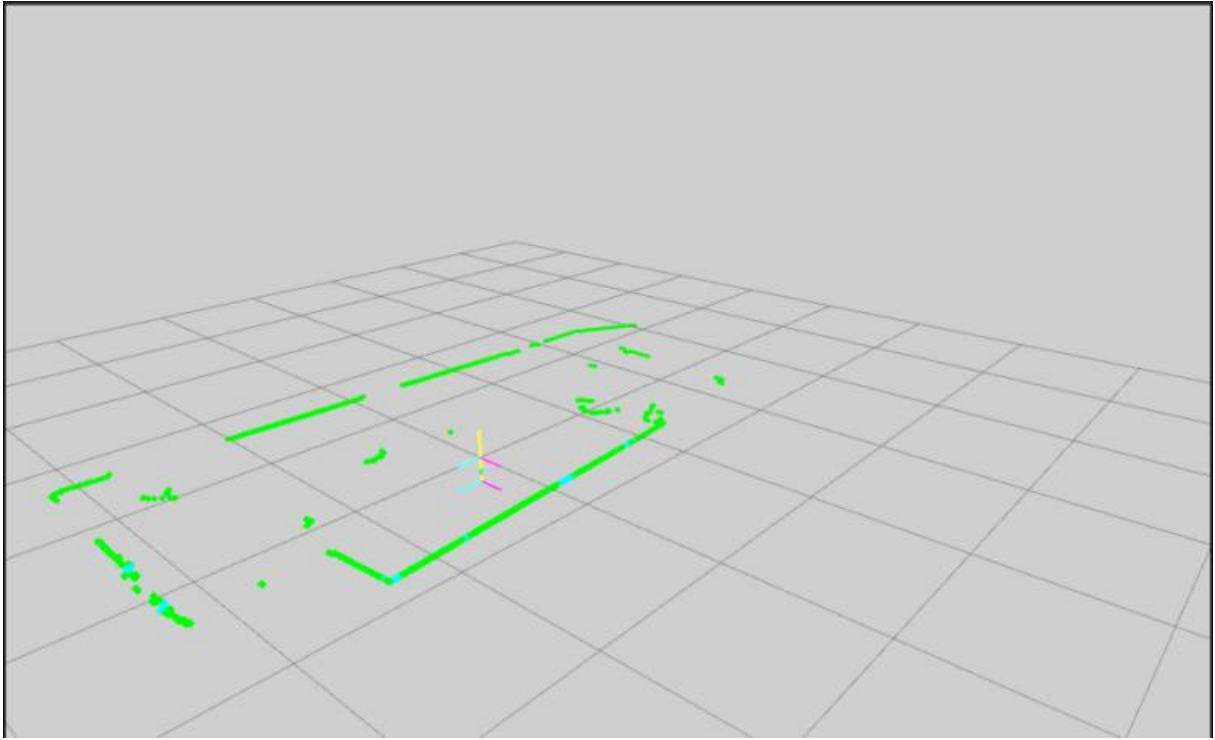
Trong đề tài, cảm biến LiDAR A2M12 được sử dụng để quét môi trường xung quanh với góc quét 360° . Dữ liệu quét được xuất bản dưới dạng bản tin LaserScan trên ROS2 và được truyền đến node SLAM Toolbox để xử lý.



Hình 30. Sơ đồ tín hiệu từ Lidar

Quá trình xây dựng bản đồ được thực hiện theo các bước sau:

Bước 1: Thu thập dữ liệu môi trường cảm biến LiDAR liên tục phát tia laser và đo khoảng cách đến các vật cản trong môi trường. Kết quả thu được là tập hợp các điểm khoảng cách theo từng góc quét.



Hình 31. Dữ liệu scan từ Lidar hiển thị trên Rviz

Bước 2: Tiền xử lý dữ liệu node SLAM Toolbox tiếp nhận dữ liệu Laser và loại bỏ các giá trị không hợp lệ. Sau đó dữ liệu được chuyển đổi sang hệ tọa độ của robot để phục vụ cho quá trình xây dựng bản đồ.

Trong ROS2, TF2 là thư viện được sử dụng để quản lý các hệ tọa độ (Frame) và các phép biến đổi giữa chúng. Do robot được cấu thành từ nhiều thành phần khác nhau như thân xe, cảm biến LiDAR và các cơ cấu chấp hành nên mỗi thành phần sẽ có một hệ tọa độ riêng. TF2 cho phép xác định vị trí và hướng tương đối giữa các hệ tọa độ này theo thời gian thực.

Việc thiết lập chính xác cây TF (TF Tree) là yêu cầu bắt buộc đối với các gói SLAM Toolbox. Nếu thiếu hoặc sai bất kỳ phép biến đổi nào, hệ thống sẽ không thể xây dựng bản đồ, xác định vị trí hoặc điều hướng robot.

Các hệ tọa độ chính bao gồm:

- + Frame map là hệ tọa độ toàn cục của môi trường được tạo bởi SLAM Toolbox là hệ tọa độ cố định của bản đồ được sử dụng để xác định vị trí tuyệt đối của robot trong môi trường sau khi hoàn thành quá trình lập bản đồ, toàn bộ bản đồ sẽ được biểu diễn trong hệ tọa độ map.

- + Frame odom là hệ tọa độ trung gian dùng để mô tả chuyển động liên tục của robot. Đặc điểm: Thay đổi liên tục theo quá trình di chuyển, hông bị gián đoạn, có thể xuất hiện sai số tích lũy theo thời gian.

Trong hệ thống này, frame odom được tạo ra từ Laser Scan Matcher kết hợp dữ liệu LiDAR.

Frame base_link được đặt tại tâm hình học của robot đây là hệ tọa độ đại diện cho thân robot và được sử dụng làm mốc tham chiếu cho các cảm biến và cơ cấu chấp hành.

Frame laser được gắn trực tiếp với cảm biến LiDAR A2M12 frame này biểu diễn vị trí thực tế của cảm biến trên robot dữ liệu LaserScan được phát ra trong hệ tọa độ laser và sau đó được chuyển đổi về các frame khác thông qua TF2.

Cấu trúc cây TF của hệ thống: map - odom - base_link - laser

Trong đó:

- + map $\rightarrow$ odom do SLAM Toolbox phát sinh.
- + odom $\rightarrow$ base_link do Laser Scan Matcher phát sinh.
- + Thiết lập phép biến đổi map $\rightarrow$ odom

Phép biến đổi map $\rightarrow$ odom được cung cấp bởi SLAM Toolbox.

Trong quá trình hoạt động, SLAM Toolbox liên tục thực hiện:

- + Thu nhận dữ liệu từ LiDAR.
- + So sánh các lần quét liên tiếp.
- + Xác định vị trí hiện tại của robot.
- + Cập nhật bản đồ môi trường.

Từ kết quả đó, node SLAM Toolbox phát bản tin TF từ frame map đến frame odom frame này cho biết robot đang nằm ở vị trí nào trên bản đồ.

Thiết lập phép biến đổi odom $\rightarrow$ base_link đây là phép biến đổi biểu diễn vị trí và hướng của robot trong không gian odometry do robot không sử dụng encoder nên không thể tính odometry bằng phương pháp truyền thống. Dữ liệu odometry được tạo ra từ Laser Scan Matcher.

Nguyên lý hoạt động:

- + LiDAR quét môi trường lần thứ nhất.
- + Robot di chuyển.
- + LiDAR quét môi trường lần thứ hai.
- + Thuật toán Scan Matching so sánh hai lần quét.
- + Tính toán độ dịch chuyển Δx , Δy và góc quay $\Delta\theta$.
- + Tạo ra bản tin odometry.

Trạng thái robot được biểu diễn bằng:

$$P = (x, y, \theta) \quad (9)$$

Trong đó:

- + x: tọa độ theo trục X.
- + y: tọa độ theo trục Y.
- + θ : góc định hướng của robot.

Giá trị này được sử dụng để tạo TF từ odom đến base_link thiết lập phép biến đổi base_link $\rightarrow$ laser. Đây là phép biến đổi tĩnh (Static Transform) do vị trí lắp đặt LiDAR trên robot không thay đổi nên chỉ cần khai báo một lần khi khởi động hệ thống.

Ví dụ:

```
ros2 run tf2_ros static_transform_publisher \
0 0 0.18 0 0 0 \
base_link laser
```

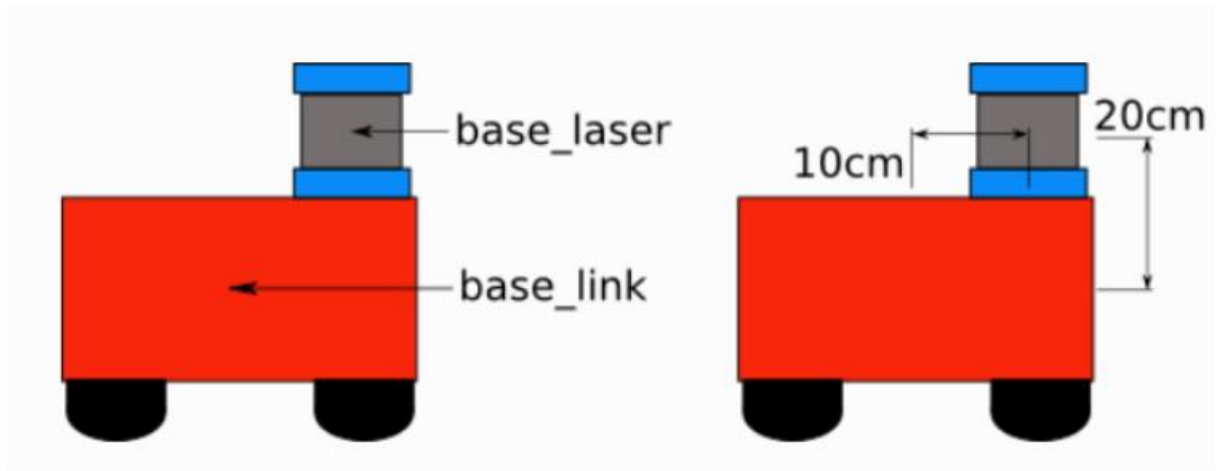
Ý nghĩa:

- + LiDAR nằm phía trước tâm robot 0 m.

+ LiDAR cao hơn mặt phẳng robot 0.18 m.

+ Không có góc xoay giữa hai hệ tọa độ.

Sau khi khởi động, ROS2 sẽ tự động phát TF này trong suốt thời gian hoạt động.



Hình 32. Mối liên hệ giữa base link và base laser

Bước 3: Scan Matching SLAM Toolbox sử dụng thuật toán Scan Matching để so sánh dữ liệu quét hiện tại với dữ liệu quét trước đó. Quá trình này giúp ước lượng sự thay đổi vị trí và hướng của robot giữa hai lần quét liên tiếp.

Scan Matching là quá trình so sánh dữ liệu quét LiDAR hiện tại với dữ liệu quét trước đó hoặc với bản đồ đã được xây dựng nhằm xác định sự thay đổi vị trí và hướng của robot trong môi trường.

Khi robot bắt đầu hoạt động, cảm biến LiDAR liên tục phát tia laser và thu thập dữ liệu khoảng cách đến các vật cản xung quanh. Mỗi lần quét sẽ tạo ra một tập hợp các điểm trong không gian biểu diễn hình dạng môi trường tại thời điểm đó.

Giả sử:

+ Scan thứ nhất được ký hiệu là (S_{t-1}) .

+ Scan thứ hai được ký hiệu là (S_t) .

Khi robot di chuyển một đoạn ngắn hoặc quay một góc nhỏ, vị trí tương đối của các điểm vật cản trong lần quét thứ hai sẽ thay đổi so với lần quét thứ nhất.

Mục tiêu của thuật toán Scan Matching là tìm ra phép biến đổi tối ưu giữa hai lần quét:

$$T = (\Delta_x, \Delta_y, \Delta_\theta) \quad (10)$$

Trong đó:

+ (Δ_x) : độ dịch chuyển theo trục X.

+ (Δ_y) : độ dịch chuyển theo trục Y.

+ (Δ_θ) : góc quay của robot.

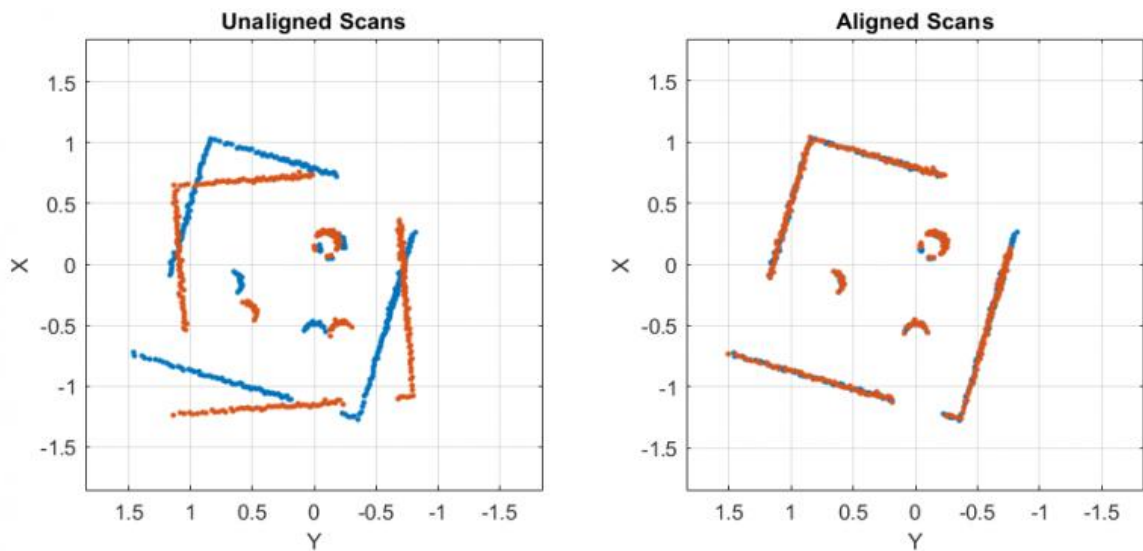
Sau khi tìm được các giá trị trên, vị trí mới của robot được cập nhật theo:

$$x_t = x_{t-1} + \Delta_x \quad (11)$$

$$y_t = y_{t-1} + \Delta_y \quad (12)$$

$$\theta_t = \theta_{t-1} + \Delta_\theta \quad (13)$$

Nhờ đó hệ thống có thể xác định được vị trí hiện tại của robot mà không cần sử dụng encoder.



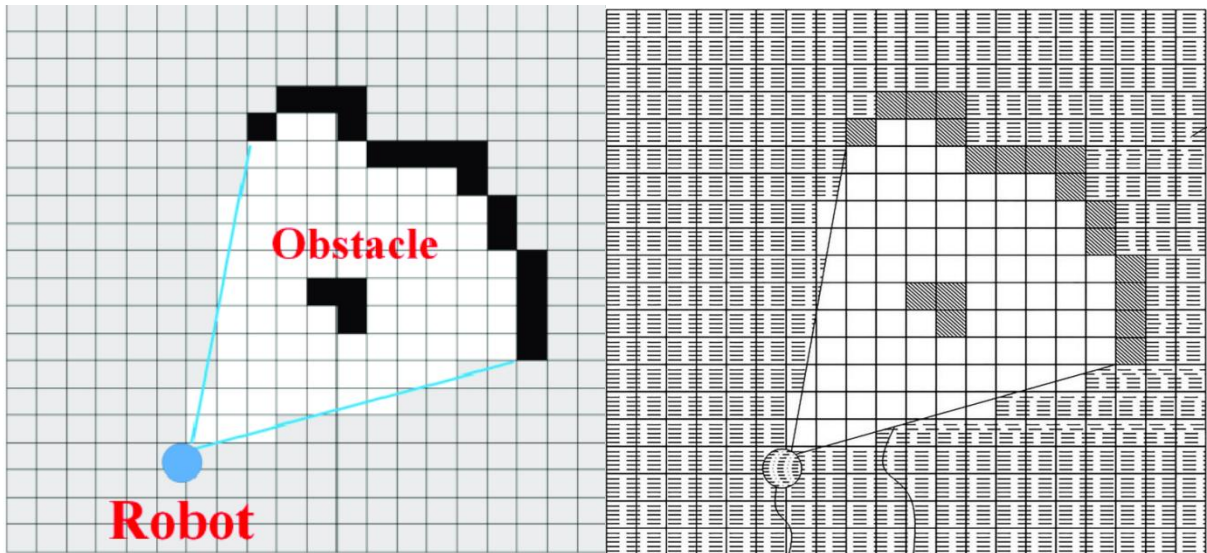
Hình 33. Bản đồ sau khi Scan Matching

Bước 4: Cập nhật bản đồ từ kết quả Scan Matching, hệ thống xác định các ô lưới có vật cản và các vùng trống trong môi trường. Các thông tin này được cập nhật liên tục lên bản đồ lưới chiếm dụng (Occupancy Grid Map).

Occupancy Grid Map là phương pháp biểu diễn môi trường bằng cách chia không gian thành nhiều ô lưới (cell) có kích thước cố định. Mỗi ô lưới lưu trữ xác suất tồn tại vật cản tại vị trí tương ứng.

Thông thường mỗi ô lưới có thể thuộc một trong ba trạng thái:

- + Ô trống (Free Space): robot có thể di chuyển qua.
- + Ô bị chiếm dụng (Occupied Space): có vật cản tồn tại.
- + Ô chưa xác định (Unknown Space): chưa được cảm biến quan sát.



Hình 34. Ô lưới trên bản đồ

Trong đó:

- + Ô đen: vật cản.
- + Ô trắng: vùng trống.
- + Ô xám: vùng chưa được quét.

Nguyên lý cập nhật bản đồ là sau khi LiDAR thực hiện quét môi trường, mỗi tia laser sẽ cung cấp:

- + Góc quét (θ).
- + Khoảng cách đo được (d).

Khoảng cách đến vật cản được xác định theo nguyên lý Time-of-Flight:

$$d = \frac{c \times t}{2} \quad (14)$$

Trong đó:

- + (d): khoảng cách đến vật cản.
- + (c): tốc độ ánh sáng.
- + (t): thời gian truyền và nhận tín hiệu laser.

Từ vị trí hiện tại của robot (x_r, y_r, θ_r), tọa độ điểm va chạm của tia laser được xác định bởi:

$$x = x_r + \frac{d}{\cos(\theta_r + \theta)} \quad (15)$$

$$y = y_r + \frac{d}{\sin(\theta_r + \theta)} \quad (16)$$

Các điểm này được chiếu lên bản đồ lưới để xác định vị trí vật cản.

Cập nhật ô lưới

Đối với mỗi tia laser:

- + Các ô nằm giữa robot và vật cản được đánh dấu là vùng trống (Free).
- + Ô tại vị trí va chạm cuối cùng được đánh dấu là có vật cản (Occupied).

Khi robot tiếp tục di chuyển và thực hiện các lần quét mới, thông tin từ các lần quét được tích hợp liên tục vào bản đồ hiện có. Điều này giúp mở rộng phạm vi bản đồ và tăng độ chính xác của môi trường được xây dựng.

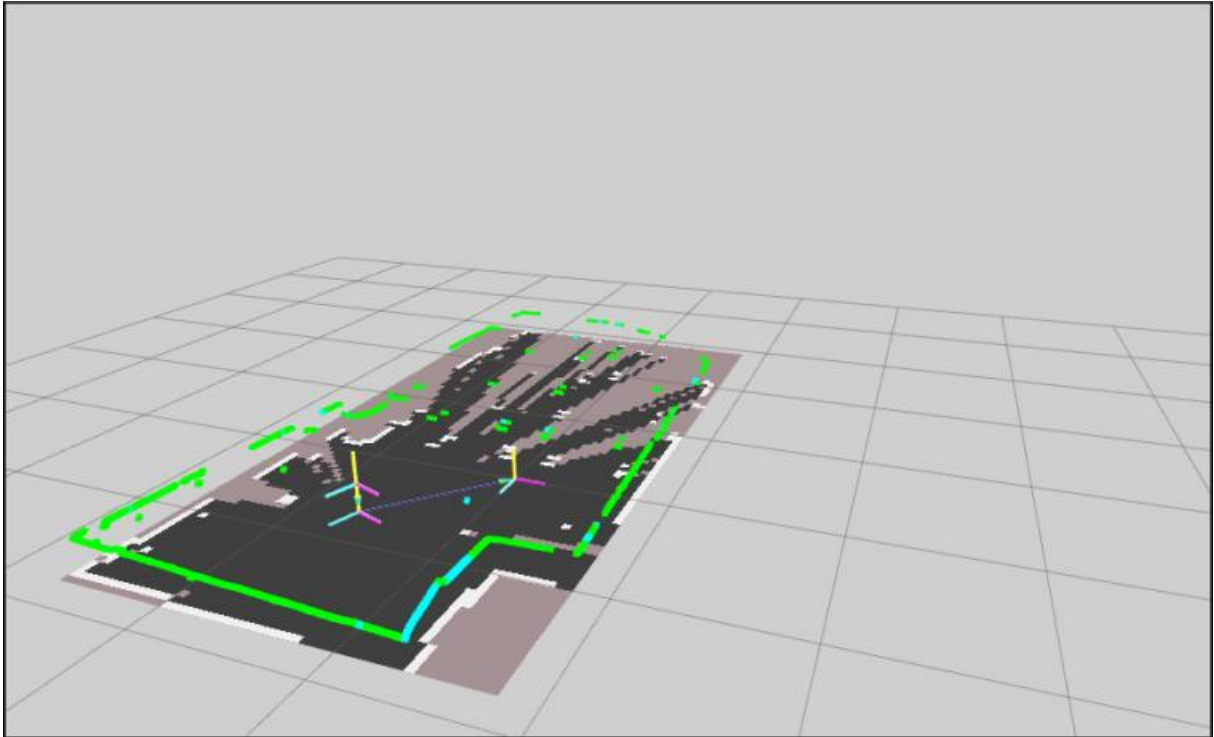
Vai trò của Occupancy Grid Map là cơ sở dữ liệu môi trường cho toàn bộ hệ thống điều hướng, bản đồ này được sử dụng để:

- + Hiển thị môi trường trên RViz.
- + Xác định vị trí robot.
- + Phát hiện vật cản.
- + Lập kế hoạch đường đi.
- + Hỗ trợ thuật toán điều hướng của Nav2.

Bước 5: Tối ưu hóa bản đồ SLAM Toolbox thực hiện tối ưu hóa đồ thị pose graph nhằm giảm sai số tích lũy trong quá trình di chuyển. Khi robot quay lại những khu vực đã đi qua trước đó, thuật toán sẽ thực hiện phát hiện vòng lặp (Loop Closure) để hiệu chỉnh lại bản đồ và vị trí robot.

Nhờ khả năng tối ưu hóa pose graph và phát hiện loop closure, SLAM Toolbox tạo ra bản đồ có độ chính xác cao hơn, đồng thời giảm đáng kể sai số tích lũy trong quá trình hoạt động lâu dài của robot.

Sau khi hoàn thành quá trình xây dựng bản đồ, dữ liệu bản đồ được lưu dưới dạng tệp YAML và PGM để phục vụ cho quá trình định vị và điều hướng robot trong các giai đoạn tiếp theo.



Hình 35. Bản đồ quét được Lidar

3.4. Lưu đồ thuật toán

Nguyên lý các thức vận hành:

Hệ thống robot chữa cháy được thiết kế với hai chế độ hoạt động gồm chế độ điều khiển thủ công (Manual Mode) và chế độ tự động (Auto Mode). Toàn bộ quá trình hoạt động được quản lý bởi Raspberry Pi 4 và ESP32.

Khi cấp nguồn cho hệ thống, Raspberry Pi 4 khởi động hệ điều hành Ubuntu và các node ROS2 cần thiết, bao gồm driver LiDAR, SLAM Toolbox và các node điều khiển. Đồng thời, ESP32 tiến hành khởi tạo các chân điều khiển động cơ, servo, cảm biến lửa, relay điều khiển bơm nước và kết nối Bluetooth với tay cầm điều khiển.

Sau khi hoàn tất quá trình khởi tạo, ESP32 chờ kết nối từ tay cầm Bluetooth. Khi tay cầm được kết nối thành công, hệ thống sẽ tiếp nhận các lệnh điều khiển từ người vận hành và cho phép lựa chọn giữa chế độ thủ công hoặc chế độ tự động.

Chế độ điều khiển thủ công (Manual Mode)

Trong chế độ thủ công, người vận hành sử dụng tay cầm Bluetooth để điều khiển trực tiếp chuyển động của robot. Các tín hiệu từ các nút điều khiển trên tay cầm được ESP32 tiếp nhận và chuyển thành lệnh điều khiển động cơ.

Robot có thể thực hiện các thao tác:

Di chuyển tiến.

Di chuyển lùi.

Rẽ trái.

Rẽ phải.

Trong chế độ này, mọi chuyển động của robot đều do người vận hành quyết định. Chế độ thủ công được sử dụng để kiểm tra hệ thống, di chuyển robot đến vị trí mong muốn hoặc xử lý các tình huống mà chế độ tự động gặp khó khăn.

Chế độ tự động (Auto Mode)

Khi chuyển sang chế độ tự động, Raspberry Pi 4 sẽ đảm nhiệm việc xử lý dữ liệu cảm biến và điều hướng robot.

Cảm biến LiDAR A2M12 liên tục quét môi trường xung quanh và gửi dữ liệu khoảng cách về Raspberry Pi 4. Dữ liệu này được xử lý bởi SLAM Toolbox nhằm xây dựng bản đồ môi trường và xác định vị trí hiện tại của robot.

Thông qua thuật toán Scan Matching, hệ thống so sánh các lần quét LiDAR liên tiếp để ước lượng sự thay đổi vị trí và hướng của robot. Kết quả định vị được sử dụng để xây dựng bản đồ lưới chiếm dụng (Occupancy Grid Map) và phục vụ cho việc điều hướng trong các môi trường trong nhà với các chướng ngại vật phức tạp có trong đó.

Trong quá trình di chuyển, robot thực hiện thuật toán bám tường kết hợp tránh vật cản. Khoảng cách đến các vật thể xung quanh được tính toán từ dữ liệu LiDAR, từ đó xác định hướng di chuyển phù hợp. Bộ điều khiển PID được sử dụng để duy trì khoảng cách ổn định giữa robot và tường, giúp robot di chuyển mượt mà và hạn chế dao động, đồng thời nâng cao độ chính xác trong quá trình điều hướng di chuyển.

Các lệnh vận tốc được Raspberry Pi 4 gửi tới ESP32 thông qua giao tiếp UART. ESP32 tiếp nhận các lệnh này và điều khiển module công suất L298N để dẫn động hai động cơ DC đảm bảo robot di chuyển theo đúng quỹ đạo và tốc độ yêu cầu.

Trong suốt quá trình hoạt động, cảm biến lửa liên tục giám sát môi trường.

Khi cảm biến phát hiện ngọn lửa, tín hiệu được gửi tới ESP32. Bộ điều khiển sẽ kích hoạt relay đóng nguồn cho động cơ bơm nước 12V.

Nước được bơm qua hệ thống vòi phun để thực hiện quá trình dập tắt đám cháy.

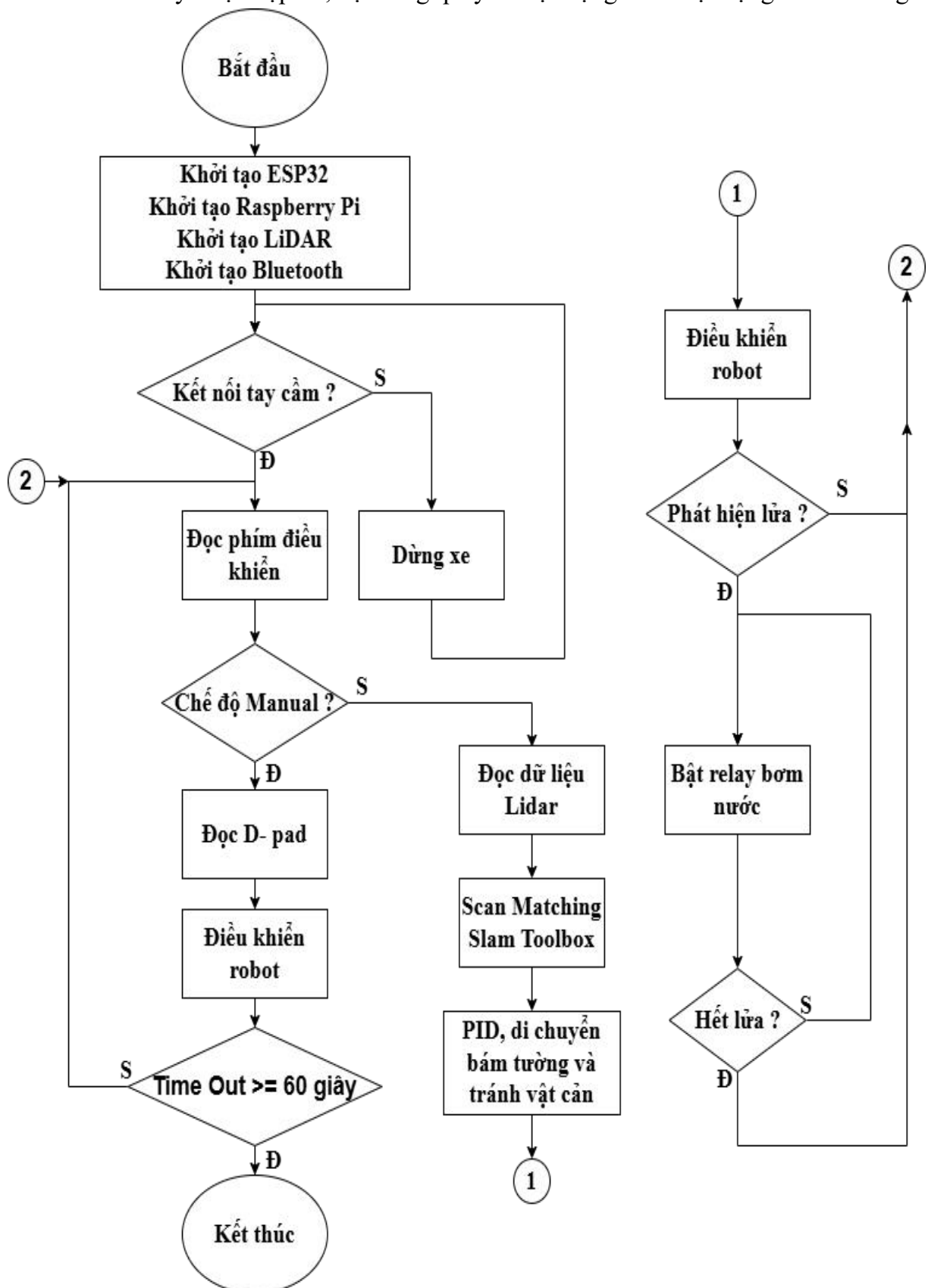
Nếu ngọn lửa vẫn còn tồn tại, hệ thống tiếp tục duy trì hoạt động của bơm nước. Khi cảm biến không còn phát hiện lửa, relay sẽ được ngắt, động cơ bơm dừng hoạt động và robot quay trở lại trạng thái hoạt động bình thường.

Ngoài ra, hệ thống còn hỗ trợ khả năng giám sát trạng thái hoạt động theo thời gian thực thông qua dữ liệu từ các cảm biến và module điều khiển. Việc kết hợp giữa Raspberry Pi 4 và ESP32 giúp phân chia nhiệm vụ xử lý hợp lý, nâng cao hiệu suất hoạt động của toàn hệ thống. Raspberry Pi đảm nhiệm các tác vụ tính toán phức tạp như SLAM và điều hướng, trong khi ESP32 thực hiện điều khiển thời gian thực đối với động cơ và các cơ cấu chấp hành. Nhờ đó, robot có thể hoạt động ổn định, phản hồi nhanh với các thay đổi của môi trường và nâng cao hiệu quả chữa cháy tự động. Mô hình được thiết kế theo hướng mở rộng, cho phép tích hợp thêm các cảm biến hoặc thuật toán điều khiển thông minh trong các nghiên cứu và phát triển sau này.

Bên cạnh đó, hệ thống còn có khả năng hoạt động liên tục nhờ sử dụng nguồn pin sạc lithium dung lượng cao. Các module được thiết kế và bố trí nhỏ gọn giúp robot dễ dàng di chuyển trong không gian hẹp của môi trường trong nhà. Toàn bộ quá trình vận hành của robot được thực hiện theo trình tự:

1. Khởi tạo Raspberry Pi 4, ESP32, LiDAR và Bluetooth.
2. Kết nối tay cầm điều khiển.
3. Lựa chọn chế độ Manual hoặc Auto.
4. Trong chế độ Manual, robot di chuyển theo lệnh từ tay cầm.
5. Trong chế độ Auto, robot sử dụng SLAM Toolbox để xây dựng bản đồ.

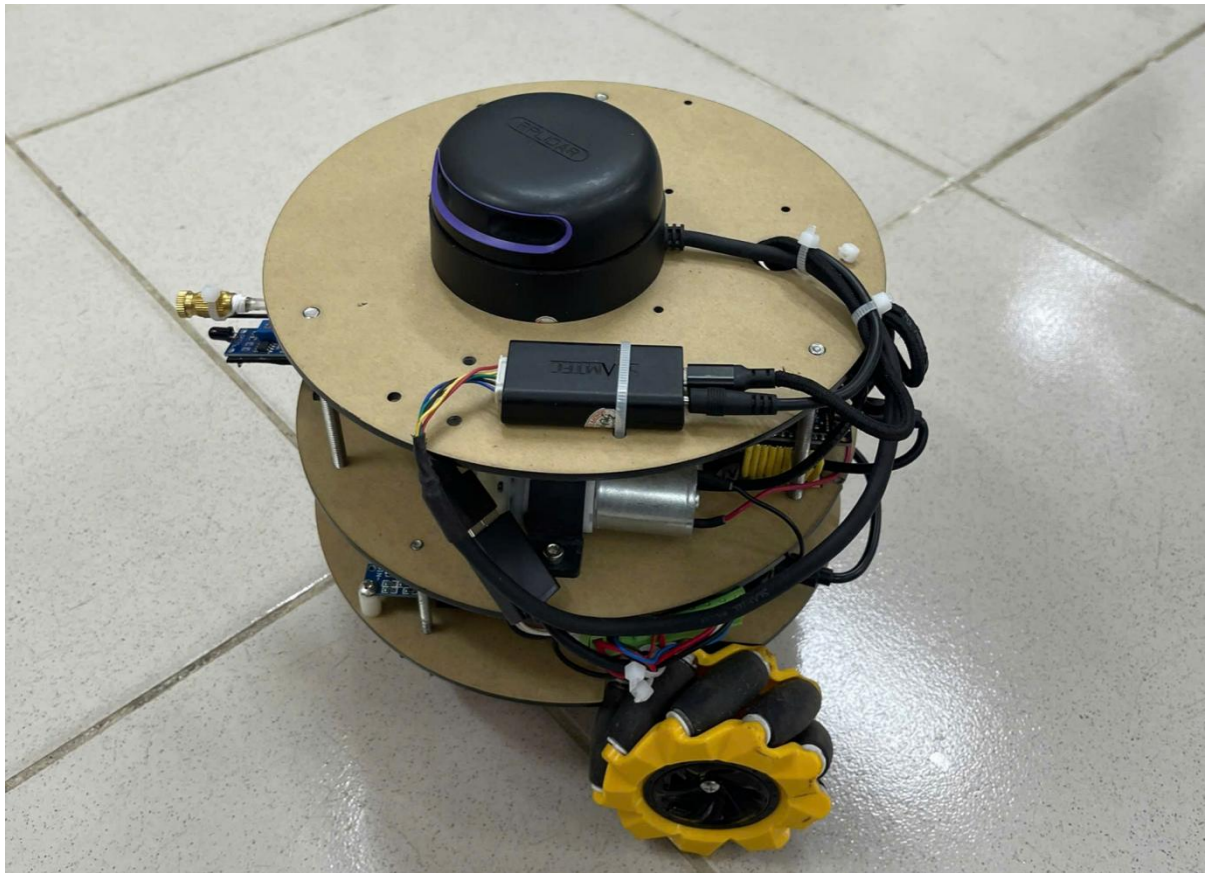
6. Thực hiện bám tường và tránh vật cản trong quá trình di chuyển.
7. Liên tục giám sát tín hiệu từ cảm biến lửa.
8. Khi phát hiện cháy, kích hoạt relay và bơm nước để dập lửa.
9. Đám cháy được dập tắt, hệ thống quay trở lại trạng thái hoạt động bình thường.



Hình 36. Lưu đồ thuật toán

3.5. Kết quả xây dựng mô hình hệ thống xe tự hành cứu hỏa

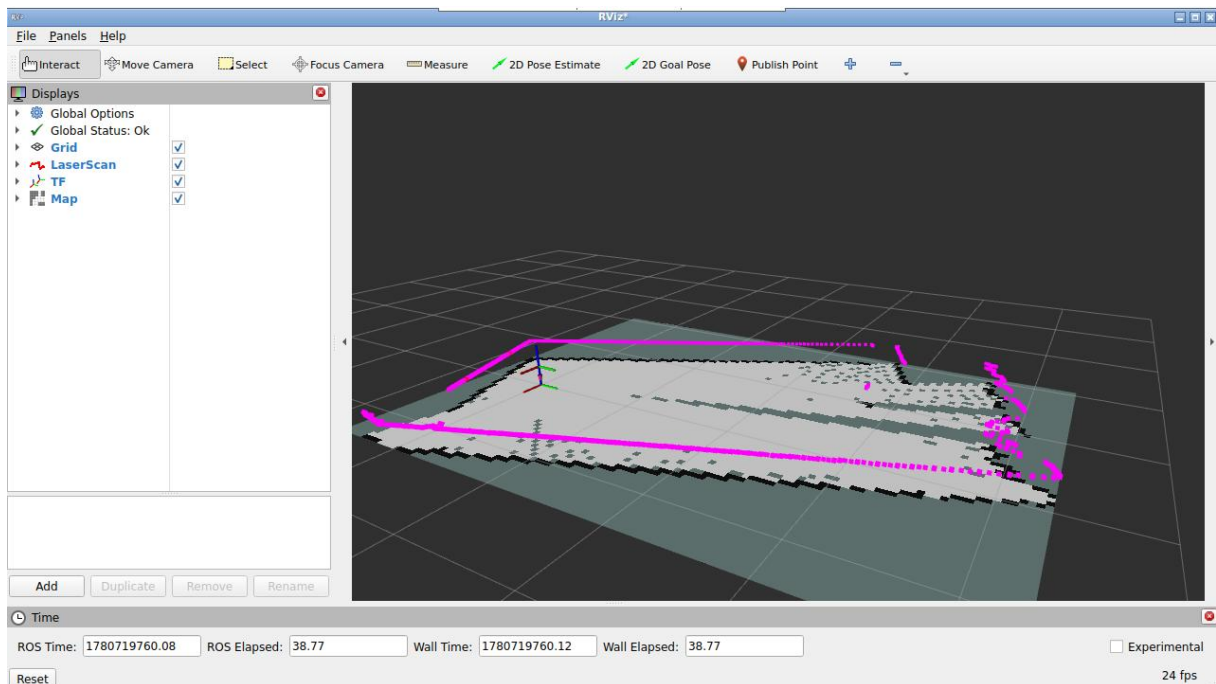
3.5.1. Kết quả xây dựng mô hình phần cứng



Hình 37. Hình ảnh kết quả xây dựng mô hình phần cứng robot



Hình 38. Hình ảnh khi robot phát hiện ngọn lửa



Hình 39. Hình ảnh kết quả xây dựng bản đồ của robot trên Rviz2

Mô hình phần cứng của hệ thống xe tự hành cứu hỏa được xây dựng bao gồm các thành phần chính như bộ điều khiển trung tâm, hệ thống cảm biến, hệ thống truyền động và hệ thống chữa cháy. Trong đó, hệ thống cảm biến sử dụng các cảm biến phát hiện lửa, cảm biến LiDAR và hệ thống tự hành được thực hiện thông qua các động cơ DC kết hợp driver điều khiển.

3.5.2. Kết quả thực nghiệm

Sau khi hoàn thành quá trình thiết kế, lắp ráp và lập trình hệ thống, nhóm tiến hành thực nghiệm nhằm đánh giá khả năng hoạt động của robot trong các điều kiện thực tế. Các nội dung thực nghiệm bao gồm: bám tường tự động, tránh vật cản, phát hiện đám cháy và thực hiện dập tắt đám cháy.

Nhận xét kết quả:

Kết quả thực nghiệm cho thấy robot có khả năng xây dựng bản đồ môi trường bằng cảm biến LiDAR A2M12 kết hợp với SLAM Toolbox trên nền tảng ROS2. Trong điều kiện môi trường có diện tích nhỏ, ít vật cản và cấu trúc tương đối đơn giản, hệ thống tạo được bản đồ tương đối chính xác, các bức tường và vật thể được thể hiện rõ trên giao diện RViz. Tuy nhiên, khi môi trường có diện tích lớn hoặc xuất hiện nhiều vật cản phức tạp, chất lượng bản đồ giảm do sai số tích lũy trong quá trình định vị và hạn chế của thuật toán điều hướng hiện tại.

Trong quá trình di chuyển tự động, robot đã thực hiện được chức năng bám tường dựa trên dữ liệu từ cảm biến LiDAR. Hệ thống có thể nhận biết khoảng cách đến tường và điều chỉnh hướng di chuyển để duy trì quỹ đạo theo biên tường. Tuy nhiên, độ ổn định của quá trình bám tường chưa thực sự cao, robot vẫn xuất hiện hiện tượng dao động và thay đổi khoảng cách đến tường tại một số vị trí, đặc biệt là khi đi qua các góc cua hoặc khu vực có nhiều bề mặt phản xạ tín hiệu LiDAR.

Bên cạnh đó, chức năng tránh vật cản hoạt động tương đối hiệu quả. Robot có khả năng phát hiện các vật cản nằm trên đường di chuyển và tự động thay đổi hướng

để tránh va chạm. Trong các lần thử nghiệm, robot không xảy ra va chạm với vật cản và vẫn tiếp tục thực hiện nhiệm vụ sau khi hoàn thành thao tác tránh vật cản.

Đối với chức năng chữa cháy, cảm biến lửa đã phát hiện được nguồn cháy mô phỏng trong phạm vi hoạt động của hệ thống. Sau khi phát hiện ngọn lửa, ESP32 kích hoạt relay điều khiển động cơ bơm nước để thực hiện quá trình chữa cháy. Kết quả cho thấy hệ thống có khả năng phát hiện đám cháy nhanh chóng, kích hoạt cơ cấu chữa cháy chính xác và dập tắt thành công nguồn cháy trong các lần thử nghiệm.

Nhìn chung, mô hình robot đã hoàn thành được các mục tiêu cơ bản của đề tài như xây dựng bản đồ môi trường, điều khiển tự động, bám tường, tránh vật cản và chữa cháy tự động. Mặc dù hệ thống vẫn còn một số hạn chế về độ ổn định khi bám tường và khả năng lập bản đồ trong môi trường lớn hoặc phức tạp, kết quả thực nghiệm đã chứng minh được tính khả thi của việc ứng dụng Raspberry Pi 4, ESP32, LiDAR và ROS2 trong thiết kế và chế tạo mô hình robot tự hành phát hiện và dập tắt đám cháy.

KẾT LUẬN KIẾN NGHỊ VÀ HƯỚNG PHÁT TRIỂN

Kết quả thu được từ đề tài

Sau thời gian nghiên cứu, thiết kế và chế tạo, đề tài “Nghiên cứu, thiết kế và chế tạo mô hình xe robot tự hành phát hiện và dập tắt đám cháy sử dụng ESP32, Raspberry Pi 4 và cảm biến LiDAR” đã hoàn thành các mục tiêu đề ra và đạt được những kết quả sau:

- Thiết kế và chế tạo thành công mô hình xe robot tự hành sử dụng Raspberry Pi 4 làm bộ xử lý trung tâm và ESP32 làm bộ điều khiển cơ cấu chấp hành.
- Xây dựng được hệ thống điều khiển chuyển động cho robot sử dụng hai động cơ DC, cho phép robot di chuyển tiến, lùi, quay trái và quay phải theo yêu cầu.
- Tích hợp thành công cảm biến LiDAR A2M12 vào hệ thống ROS2 để thu thập dữ liệu môi trường.
- Xây dựng và vận hành thành công hệ thống lập bản đồ môi trường bằng gói SLAM Toolbox trên nền tảng ROS2.
- Thiết lập được hệ tọa độ TF và cơ chế Scan Matching để xác định vị trí và hướng của robot trong quá trình hoạt động.
- Xây dựng được bản đồ môi trường dưới dạng Occupancy Grid Map phục vụ cho quá trình định vị.
- Thiết kế và tích hợp hệ thống phát hiện đám cháy sử dụng cảm biến lửa.
- Xây dựng cơ cấu chữa cháy gồm bơm nước 12V, relay điều khiển và vòi phun nước.
- Thực hiện thành công việc phát hiện nguồn cháy và kích hoạt hệ thống phun nước để dập tắt đám cháy trong phạm vi thử nghiệm.
- Thiết lập giao tiếp UART giữa Raspberry Pi 4 và ESP32, đảm bảo việc truyền nhận dữ liệu điều khiển ổn định.
- Hoàn thiện mô hình robot chữa cháy có khả năng tự hành, nhận biết môi trường và thực hiện nhiệm vụ chữa cháy tự động.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định, đáp ứng được các yêu cầu cơ bản của đề tài. Robot có khả năng xây dựng bản đồ môi trường, tránh vật cản, phát hiện đám cháy và thực hiện quá trình chữa cháy tự động trong môi trường thử nghiệm.

Kiến nghị, hướng phát triển của đề tài

Mặc dù đã đạt được các mục tiêu đề ra, đề tài vẫn còn một số hạn chế về thời gian nghiên cứu, kinh phí và điều kiện thử nghiệm. Trong tương lai, đề tài có thể được tiếp tục phát triển theo các hướng sau:

- Tích hợp thêm camera và các thuật toán xử lý ảnh, trí tuệ nhân tạo để nâng cao khả năng phát hiện và nhận dạng đám cháy.
- Sử dụng các cảm biến nhiệt độ, cảm biến khí gas hoặc camera nhiệt nhằm tăng độ chính xác trong việc phát hiện cháy.
- Tích hợp hệ thống điều hướng Nav2 để thực hiện lập kế hoạch đường đi tối ưu và tự động tiếp cận vị trí đám cháy trong môi trường phức tạp.

- Bổ sung encoder hoặc IMU để cải thiện độ chính xác của quá trình định vị và điều khiển chuyển động.

- Nâng cấp cơ cấu chữa cháy với khả năng điều khiển góc phun tự động bằng servo để tăng hiệu quả dập lửa.

- Xây dựng hệ thống giám sát và điều khiển từ xa thông qua mạng WiFi hoặc Internet.

- Tối ưu hóa thuật toán điều hướng nhằm giảm thời gian tiếp cận đám cháy và nâng cao hiệu quả hoạt động.

- Nâng cấp nguồn năng lượng và cơ cấu truyền động để tăng thời gian hoạt động liên tục của robot.

- Nghiên cứu áp dụng các thuật toán học máy và trí tuệ nhân tạo trong việc nhận dạng môi trường, phân loại đám cháy và đưa ra phương án xử lý phù hợp.

- Mở rộng quy mô hệ thống từ mô hình thử nghiệm sang các ứng dụng thực tế trong nhà kho, nhà xưởng, khu công nghiệp hoặc các khu vực có nguy cơ cháy nổ cao.

Do điều kiện thời gian và kiến thức còn nhiều hạn chế nên đề tài đồ án tốt nghiệp không tránh khỏi sai sót, em mong nhận được sự góp ý, chỉ bảo từ quý thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn.

TÀI LIỆU THAM KHẢO

- [1] Open Robotics, “ROS 2 Documentation,” Available:
<https://docs.ros.org/en/humble/>
- [2] Open Robotics, “TF2 Tutorials,” Available:
<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html>2018.
- [3] Steve Macenski et al., “SLAM Toolbox,” GitHub Repository, Available:
https://github.com/SteveMacenski/slam_toolbox
- [4] Open Navigation LLC, “Navigation2 Documentation,” Available:
<https://docs.nav2.org>
- [5] Open Robotics, “OccupancyGrid Message Definition,” Available:
https://docs.ros.org/en/humble/p/nav_msgs/msg/OccupancyGrid.html
- [6] Open Robotics, “LaserScan Message Definition,” Available:
https://docs.ros.org/en/humble/p/sensor_msgs/msg/LaserScan.html
- [7] Slamtec, “RPLIDAR A2 Datasheet,” Available:
<https://www.slamtec.com/en/Lidar/A2>
- [8] Sebastian Thrun, Wolfram Burgard, Dieter Fox,
Probabilistic Robotics,
MIT Press, 2005.

PHỤ LỤC

```
#include <Bluepad32.h>
#include <ESP32Servo.h>

// ===== CẤU HÌNH =====
#define USB_UART_BAUD 115200
#define STICK_DEADZONE 20
#define STICK_CHANGE_THRESH 8
#define TRIGGER_CHANGE_THRESH 8

// ===== ĐỘNG CƠ DC =====
#define ENB 12
#define IN3 26
#define IN4 27

#define ENA 25
#define IN1 14
#define IN2 13

#define PWM_FREQ 1000
#define PWM_RES 8

#define CH_A 0
#define CH_B 1

// ===== CẢM BIẾN LỬA & RELAY BƠM =====
#define FLAME_PIN 32
#define RELAY_PIN 33

// ===== THAM SỐ PID CHO AUTO =====
double Kp = 1.5;
double Ki = 0.05;
double Kd = 1.8;

double setpoint = 40;

// Biến lưu trạng thái PID
double error = 0;
double lastError = 0;
double integral = 0;
double derivative = 0;
double output = 0;
```

```

// Giới hạn
double integralLimit = 150;
double outputLimit = 250;

unsigned long lastPIDTime = 0;

int autoBaseSpeed = 170;
int minSpeed = 100;
int maxTurnSpeed = 255;

// ===== THAM SỐ TRÁNH VẬT CẢN =====
int obstacleDistance = 35;
int dangerDistance = 24;
unsigned long obstacleTime = 0;
bool isAvoiding = false;
int avoidStep = 0;
unsigned long lastAvoidTime = 0;

// ===== THAM SỐ CẢM BIẾN LỬA =====
int flameThreshold = 40;
unsigned long flameReadTime = 0;
unsigned long lastFlameSendTime = 0;
bool fireActive = false;
unsigned long fireStartTime = 0;

// ===== Biến nhận dữ liệu từ Serial =====
int goc0, kc0;
int goc90, kc90;
int goc180, kc180;
String data = "";

// ===== Biến cho tay cầm =====
ControllerPtr connectedPad = nullptr;
uint16_t lastButtons = 0xFFFF;
uint8_t lastDpad = 0xFF;
uint8_t lastMisc = 0xFF;
int lastLX = 99999;
int lastLY = 99999;
int lastRX = 99999;
int lastRY = 99999;
int lastL2 = 99999;

```

```

int lastR2 = 99999;
bool waitingPrinted = false;

// ===== CHẾ ĐỘ: au=1(auto), ma=1(manual) =====
int au = 0, ma = 1;

// ===== MAC tay cầm được phép =====
const uint8_t ALLOWED_MAC[6] = {0x98, 0xB6, 0x20, 0xF6, 0x2D, 0xCA};

// ===== HÀM ĐỌC CẢM BIẾN LỬA =====
int readFlameFiltered() {
    int sum = 0;
    for (int i = 0; i < 5; i++) {
        sum += analogRead(FLAME_PIN);
        delay(1);
    }
    return sum / 5;
}

// ===== GỬI TÍN HIỆU LỬA LÊN PI =====
void sendFireStatus() {
    unsigned long now = millis();

    if (now - flameReadTime >= 100) {
        int flameValue = readFlameFiltered();
        flameReadTime = now;

        // Phát hiện lửa
        if (flameValue < flameThreshold && !fireActive) {
            fireActive = true;
            fireStartTime = now;
            Serial.println("FIRE");
            digitalWrite(RELAY_PIN, HIGH); // Bật bom
            stop(); // DỪNG XE NGAY LẬP TỨC
            Serial.println("!!! PHAT HIEN LUA - XE DUNG DE DAP !!!");
        }
        // Hết lửa
        else if (flameValue >= flameThreshold && fireActive) {
            fireActive = false;
            Serial.println("CLEAR");
            digitalWrite(RELAY_PIN, LOW); // Tắt bom
            Serial.println("=== HET LUA - XE CO THE CHAY LAI ===");
        }
    }
}

```

```

}

// Gửi tín hiệu lên Pi mỗi 500ms nếu đang cháy
if (fireActive && now - lastFlameSendTime >= 500) {
    Serial.println("FIRE");
    lastFlameSendTime = now;
}

// Debug
static unsigned long lastDebug = 0;
if (now - lastDebug > 2000) {
    Serial.print("Flame sensor: ");
    Serial.println(flameValue);
    Serial.print("Fire active: ");
    Serial.println(fireActive ? "YES - XE DUNG" : "NO");
    lastDebug = now;
}
}
}

// ===== HÀM PID =====
double computePID(int currentDistance) {
    unsigned long now = micros();
    float dt = (now - lastPIDTime) / 1000000.0;

    if (dt < 0.005) dt = 0.005;

    error = setpoint - currentDistance;

    double P = Kp * error;

    integral += error * dt;
    integral = constrain(integral, -integralLimit, integralLimit);
    double I = Ki * integral;

    derivative = (error - lastError) / dt;
    double D = Kd * derivative;

    output = P + I + D;
    output = constrain(output, -outputLimit, outputLimit);

    lastError = error;
}

```

```

lastPIDTime = now;

static unsigned long lastDebug = 0;
if (millis() - lastDebug > 500) {
    Serial.print("PID: Dist=");
    Serial.print(currentDistance);
    Serial.print(" | E=");
    Serial.print(error);
    Serial.print(" | P=");
    Serial.print(P);
    Serial.print(" | I=");
    Serial.print(I);
    Serial.print(" | D=");
    Serial.print(D);
    Serial.print(" | Out=");
    Serial.println(output);
    lastDebug = millis();
}

return output;
}

// ===== TRÁNH VẬT CẢN =====
void handleObstacleAvoidance() {
    // KHÔNG tránh vật cản khi đang có lửa
    if(fireActive) return;

    if(kc0 > 0 && kc0 < obstacleDistance && !isAvoiding && au == 1 && !fireActive)
    {
        isAvoiding = true;
        avoidStep = 0;
        obstacleTime = millis();
        Serial.print(">>> PHAT HIEN VAT CAN: ");
        Serial.print(kc0);
        Serial.println("cm");
    }

    if(isAvoiding && au == 1 && !fireActive) {
        unsigned long now = millis();

        switch(avoidStep) {
            case 0:

```

```

Serial.println("B1: LUI");
lui(180);
if(now - obstacleTime > 500) {
    avoidStep = 1;
    obstacleTime = now;
}
break;

case 1:
Serial.println("B2: RE TRAI");
turnLeft(200);
if(now - obstacleTime > 700) {
    avoidStep = 2;
    obstacleTime = now;
}
break;

case 2:
Serial.println("B3: TIEN");
tien(150);
if(now - obstacleTime > 300) {
    avoidStep = 3;
    obstacleTime = now;
}
break;

case 3:
if(kc0 > obstacleDistance || kc0 == 0) {
    isAvoiding = false;
    Serial.println(">>> HET VAT CAN");
} else {
    avoidStep = 0;
    obstacleTime = now;
}
break;
}
lastAvoidTime = now;
}
}

// ===== ĐIỀU KHIỂN ĐỘNG CƠ =====
void motorLeft(int speedVal, bool forwardDir) {

```

```

    speedVal = constrain(speedVal, 0, 255);
    digitalWrite(IN1, forwardDir ? HIGH : LOW);
    digitalWrite(IN2, forwardDir ? LOW : HIGH);
    ledcWrite(CH_A, speedVal);
}

void motorRight(int speedVal, bool forwardDir) {
    speedVal = constrain(speedVal, 0, 255);
    digitalWrite(IN3, forwardDir ? HIGH : LOW);
    digitalWrite(IN4, forwardDir ? LOW : HIGH);
    ledcWrite(CH_B, speedVal);
}

void turnLeft(int speedVal) {
    speedVal = constrain(speedVal, 0, 255);
    motorLeft(speedVal, false);
    motorRight(speedVal, true);
}

void turnRight(int speedVal) {
    speedVal = constrain(speedVal, 0, 255);
    motorLeft(speedVal, true);
    motorRight(speedVal, false);
}

void tien(int speedVal) {
    speedVal = constrain(speedVal, 0, 255);
    motorLeft(speedVal, true);
    motorRight(speedVal, true);
}

void lui(int speedVal) {
    speedVal = constrain(speedVal, 0, 255);
    motorLeft(speedVal, false);
    motorRight(speedVal, false);
}

void trai(int speedVal) {
    speedVal = constrain(speedVal, 0, 255);
    motorLeft(speedVal, false);
    motorRight(speedVal, true);
}

```

```

void phai(int speedVal) {
    speedVal = constrain(speedVal, 0, 255);
    motorLeft(speedVal, true);
    motorRight(speedVal, false);
}

void stop() {
    ledcWrite(CH_A, 0);
    ledcWrite(CH_B, 0);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
}

// ===== KIỂM TRA NGUY HIỂM =====
bool checkDanger() {
    return (kc0 > 0 && kc0 < dangerDistance);
}

// ===== ĐIỀU KHIỂN PID AUTO =====
void handlePIDControl() {
    // DỪNG PID NẾU ĐANG CÓ LỬA
    if(fireActive) {
        stop();
        return;
    }

    int currentDistance = kc90;

    if (currentDistance <= 0 || currentDistance >= 300) {
        stop();
        Serial.println("PID ERROR: Không có dữ liệu tin cậy");
        return;
    }

    double steering = computePID(currentDistance);

    if (abs(error) > 50) {
        int turnSpeed = 200;

```

```

    if (error > 0) {
        Serial.println("EMERGENCY: Quay trai gap");
        turnLeft(turnSpeed);
    } else {
        Serial.println("EMERGENCY: Quay phai gap");
        turnRight(turnSpeed);
    }
    delay(50);
    return;
}

int leftSpeed, rightSpeed;

leftSpeed = autoBaseSpeed - steering;
rightSpeed = autoBaseSpeed + steering;

leftSpeed = constrain(leftSpeed, minSpeed, maxTurnSpeed);
rightSpeed = constrain(rightSpeed, minSpeed, maxTurnSpeed);

motorLeft(leftSpeed, true);
motorRight(rightSpeed, true);

static unsigned long lastDebug = 0;
if (millis() - lastDebug > 300) {
    Serial.print("AUTO: Dist=");
    Serial.print(currentDistance);
    Serial.print(" | Steering=");
    Serial.print(steering);
    Serial.print(" | L=");
    Serial.print(leftSpeed);
    Serial.print(" | R=");
    Serial.println(rightSpeed);
    lastDebug = millis();
}
}

// ===== ĐIỀU KHIỂN MANUAL =====
void handleManualControl(uint16_t buttons, uint8_t dpad) {
    // KHÔNG cho điều khiển manual khi đang có lửa
    if(fireActive) {
        stop();
        return;
    }
}

```

```

}

if (dpad == 1 && buttons == 0) {
    tien(254);
    Serial.println("MANUAL: TIEN");
}
else if (dpad == 4 && buttons == 0) {
    trai(254);
    Serial.println("MANUAL: TRAI");
}
else if (dpad == 8 && buttons == 0) {
    phai(254);
    Serial.println("MANUAL: PHAI");
}
else if (dpad == 2 && buttons == 0) {
    lui(254);
    Serial.println("MANUAL: LUI");
}
else {
    stop();
}
}

// ===== HÀM MAC =====
bool isSameMAC(const uint8_t* a, const uint8_t* b) {
    for (int i = 0; i < 6; i++) {
        if (a[i] != b[i]) return false;
    }
    return true;
}

void printMac(const uint8_t* mac) {
    for (int i = 0; i < 6; i++) {
        if (mac[i] < 16) Serial.print("0");
        Serial.print(mac[i], HEX);
        if (i < 5) Serial.print(":");
    }
}

// ===== RESET TRẠNG THÁI =====
void resetLastState() {
    lastButtons = 0xFFFF;
}

```

```

lastDpad = 0xFF;
lastMisc = 0xFF;
lastLX = 99999;
lastLY = 99999;
lastRX = 99999;
lastRY = 99999;
lastL2 = 99999;
lastR2 = 99999;
}

// ===== CALLBACK KẾT NỐI =====
void onConnectedController(ControllerPtr ctl) {
    if (connectedPad != nullptr && connectedPad != ctl) {
        ctl->disconnect();
        return;
    }

    auto props = ctl->getProperties();

    if (isSameMAC(props.btaddr, ALLOWED_MAC)) {
        connectedPad = ctl;
        waitingPrinted = false;
        Serial.println("Da ket noi PS5");
        Serial.print("MAC: ");
        printMac(props.btaddr);
        Serial.println();
        resetLastState();
    } else {
        ctl->disconnect();
    }
}

void onDisconnectedController(ControllerPtr ctl) {
    if (connectedPad == ctl) {
        connectedPad = nullptr;
        stop();
        Serial.println("Mat ket noi PS5");
        resetLastState();
        waitingPrinted = false;
    }
}

```

```

// ===== XỬ LÝ TAY CÀM =====
void processGamepad(ControllerPtr ctl) {
    if (!ctl || !ctl->isConnected()) return;

    auto props = ctl->getProperties();
    if (!isSameMAC(props.btaddr, ALLOWED_MAC)) {
        ctl->disconnect();
        connectedPad = nullptr;
        return;
    }

    uint16_t buttons = ctl->buttons();
    uint8_t dpad = ctl->dpad();
    uint8_t misc = ctl->miscButtons();

    // Chuyển chế độ: MISC 4 = AUTO, MISC 2 = MANUAL
    if(misc & 4) {
        au = 1;
        ma = 0;
        Serial.println(">>> CHUYEN SANG CHE DO AUTO (BAM TUONG + TRANH VAT CAN)");
        delay(200);
    }
    else if(misc & 2) {
        au = 0;
        ma = 1;
        Serial.println(">>> CHUYEN SANG CHE DO MANUAL (TAY CAM)");
        delay(200);
    }

    // ===== ƯU TIÊN SỐ 1: NẾU CÓ LỬA, DỪNG XE NGAY LẬP TỨC =====
    if(fireActive) {
        stop();
        // Không xử lý điều khiển xe khi đang có lửa
        return;
    }

    // Xử lý AUTO hoặc MANUAL (chỉ khi không có lửa)
    if(au == 1 && ma == 0) {
        // Đọc dữ liệu từ Serial cho Auto
        while (Serial.available()) {
            char c = Serial.read();

```

```

    if (c == '\n') {
        sscanf(data.c_str(), "%d,%d,%d,%d,%d,%d",
            &goc0, &kc0, &goc90, &kc90, &goc180, &kc180);
        data = "";
    }
    else {
        data += c;
    }
}

// === ƯU TIÊN TRÁNH VẬT CẢN ===
handleObstacleAvoidance();

// Nếu đang tránh vật cản, bỏ qua PID
if(isAvoiding) {
    delay(10);
    return;
}

// === XỬ LÝ NGUY HIỂM ===
if(checkDanger()) {
    Serial.print("!!! NGUY HIEM! DUNG KHAI - Kc0=");
    Serial.println(kc0);
    stop();
    delay(100);
    return;
}

// === ĐIỀU KHIỂN PID BẮM TƯỜNG ===
handlePIDControl();
}
else if(ma == 1 && au == 0) {
    handleManualControl(buttons, dpad);
}
}

// ===== SETUP =====
void setup() {
    Serial.begin(USB_UART_BAUD);
    Serial.println("=== HE THONG AUTO + MANUAL + CAM BIEN LUA ===");
    Serial.println("=== CHE DO: KHI PHAT HIEN LUA, XE DUNG NGAY ===");
}

```

```

// Khởi tạo chân động cơ
pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(IN3, OUTPUT);
pinMode(IN4, OUTPUT);

// Khởi tạo cảm biến lửa và relay
pinMode(FLAME_PIN, INPUT);
pinMode(RELAY_PIN, OUTPUT);

// Khởi tạo PWM
ledcSetup(CH_A, PWM_FREQ, PWM_RES);
ledcSetup(CH_B, PWM_FREQ, PWM_RES);
ledcAttachPin(ENA, CH_A);
ledcAttachPin(ENB, CH_B);

stop();
digitalWrite(RELAY_PIN, LOW);
lastPIDTime = micros();

// Khởi tạo Bluetooth
BP32.setup(&onConnectedController, &onDisconnectedController);
BP32.enableVirtualDevice(false);

Serial.println("Khoi dong thanh cong!");
Serial.println("CHE DO: MANUAL (mac dinh)");
Serial.println("MISC 2: MANUAL | MISC 4: AUTO");
Serial.println("AUTO: PID bam tuong + tranh vat can");
Serial.println("CAM BIEN LUA: Phat hien chay -> DUNG XE + BAT BOM");
Serial.println("PID: Kp=1.8, Ki=0.05, Kd=0.8, setpoint=40cm");
Serial.println("=====");
}

// ===== LOOP =====
void loop() {
    BP32.update();

    // Gửi tín hiệu cảm biến lửa lên Pi (hàm này sẽ tự động dừng xe nếu phát hiện lửa)
    sendFireStatus();

    if (connectedPad && connectedPad->isConnected()) {
        processGamepad(connectedPad);
    }
}

```

```
} else {  
  stop();  
  if (!waitingPrinted) {  
    Serial.println("Dang doi ket noi PS5...");  
    waitingPrinted = true;  
  }  
}  
}  
}
```